



Politechnika Warszawska

Wydział Matematyki
i Nauk Informatycznych

Reprezentacja Wiedzy

Procesy działań złożonych

Punkt kontrolny 2

Zespół projektowy nr 4:

Wojciech Jasiński
Krzysztof Gryboś
Jarosław Bieniek
Filip Filipkowski
Sebastian Rydz
Jakub Pietrzak
Sandra Adamiec

Warszawa, 23 kwietnia 2026

Spis treści

1	Wprowadzenie	3
1.1	Cel projektu	3
1.2	Klasa $\mathbb{DS4}$ – warunki Z1–Z7	3
1.3	Mapa zapożyczeń z wykładów	3
1.4	Ograniczenia $\mathbb{DS4}$	3
2	Podstawy formalne	4
2.1	Sygnatura	4
2.2	Formuły zdaniowe	5
2.3	Stany	5
2.4	Literały i DNF	5
3	Język akcji — składnia	5
3.1	Zdania dziedziny — tabela zbiorcza	6
3.2	Pełna BNF dziedziny	6
3.3	Opis poszczególnych zdań	7
3.4	Dziedzina D	8
4	Semantyka — poziom akcji prostej	8
4.1	Stany dopuszczalne Σ	9
4.2	Stany początkowe Σ_0	9
4.3	Funkcja Res_0	9
4.4	Funkcja New	9
4.5	Minimalizacja — $\text{Res}(A, \sigma)$	10
4.6	Niewykonalność	10
5	Semantyka — akcje złożone	11
5.1	Akcja złożona $\mathbb{A}$	11
5.2	Konflikty	11
5.3	Dekompozycje $\Delta(\mathbb{A}, \sigma)$	12
5.4	$\text{Res}(\mathbb{A}, \sigma)$ przez dekompozycje	13
5.5	Uwaga o „restricted inheritance”	14
5.6	Porównanie z treścią projektu	14
6	Procesy i model dziedziny	15
6.1	Proces $\mathbb{P}$	15
6.2	Funkcja Ψ	15
6.3	Prawdziwość zdań wartościowych	15
6.4	Model dziedziny	16
6.5	Jednoznaczność modelu	16
7	Język kwerend	17
7.1	Zdania języka kwerend	17
7.2	Składnia	17
7.3	Semantyka formalna	18
8	Algorytmizacja (pseudokod)	18
8.1	Generacja Σ	19
8.2	Res dla akcji prostej	19
8.3	Generacja Σ_0	21
8.4	Wykrywanie konfliktów	22

8.5	Generacja dekompozycji	23
8.6	Res dla akcji złożonej	24
8.7	Ewaluacja kwerend	25
8.8	Uwagi o złożoności	26
9	Decyzje projektowe i uzasadnienia	26
9.1	Σ_0 zamiast σ_0	27
9.2	Mechanizm AC zamiast rozłączności z treści projektu	27
9.3	$\text{Res}(\mathbb{A}, \sigma) = \emptyset$ przy braku wykonalnych składowych	27
9.4	Semantyka przypadków brzegowych	27
10	Przykłady	28
10.1	YSP z rosyjską ruletką	28
10.2	Producent i konsumenci	29
10.3	Dwa przełączniki, żarówka i alarm	30
10.4	Dwóch robotników wynoszących stół	32
11	Podsumowanie i kierunek dalszych prac	36

1 Wprowadzenie

1.1 Cel projektu

Zadanie polega na zaprojektowaniu dwóch formalnych języków dla klasy $\mathbb{DS4}$ systemów dynamicznych:

- *języka akcji* — służącego do opisu dziedziny (fluenty, akcje, ich skutki, ograniczenia),
- *języka kwerend* — pozwalającego zadawać pytania o wykonywalność procesów i osiągalność celów.

Klasa $\mathbb{DS4}$ łączy cechy kilku języków akcji znanych z wykładów. Mechanizmy inercji, nie-determinizmu, ramifikacji i niewykonalności pochodzą z języka AR (wykład 2). Obsługę akcji złożonych projektujemy w oparciu o język AC (wykład 5) — adaptujemy mechanizm detekcji konfliktów i dekompozycji do warunków $\mathbb{DS4}$, w których opisy dziedzin zawierają wyłącznie akcje proste.

1.2 Klasa $\mathbb{DS4}$ – warunki Z1–Z7

Systemy klasy $\mathbb{DS4}$ są zdefiniowane przez siedem fundamentalnych założeń, które określają ich semantykę i możliwości modelowania:

1. **Z1. Prawo inercji** – fluenty, które nie są bezpośrednio lub pośrednio zmieniane przez akcję, zachowują swoją wartość w kolejnym stanie (np. jeśli akcja "malowanie ściany" zmienia tylko kolor, to położenie drzwi pozostaje bez zmian).
2. **Z2. Pełna informacja o strukturze** – system zakłada dostęp do kompletnej wiedzy o zbiorze fluentów F , akcji A oraz wszystkich ich potencjalnych skutkach zdefiniowanych w modelu.
3. **Z3. Niedeterminizm działań** – wykonanie pojedynczej akcji w konkretnym stanie może prowadzić do wielu alternatywnych stanów wynikowych (np. rzut monetą skutkuje orłem lub reszką).
4. **Z4. Warunki wstępne i wynikowe** – każda akcja posiada zdefiniowaną formułę logiczną, która musi być spełniona przed jej wykonaniem; w przypadku niespełnienia warunku wstępnego, akcja nie wywołuje żadnych zmian (efekt pusty).
5. **Z5. Niewykonalność akcji** – model dopuszcza istnienie stanów, w których dana akcja jest całkowicie zablokowana i nie może zostać podjęta przez agenta.
6. **Z6. Warunki integralności (Ramifikacje)** – globalne więzy logiczne nakładane na stany wpływają na skutki uboczne akcji; zmiana jednego fluentu może wymusić zmianę innego, aby zachować spójność systemu.
7. **Z7. Opis częściowy** – język pozwala na specyfikację stanów, w których wartości niektórych fluentów są nieznane, co dotyczy zarówno stanu początkowego, jak i stanów osiągniętych po sekwencjach akcji.

1.3 Mapa zapożyczeń z wykładów

1.4 Ograniczenia $\mathbb{DS4}$

Pomimo wysokiego stopnia zaawansowania, klasa $\mathbb{DS4}$ wprowadza pewne uproszczenia względem pełnej specyfikacji języka AC oraz rozszerzonych wersji AR , co ma na celu ujednolicenie semantyki przy zachowaniu pełnej mocy wyrazowej dla procesów złożonych. Do kluczowych ograniczeń należą:

Tabela 1: Mapowanie mechanizmów klasy $\mathbb{DS4}$ na materiały teoretyczne.

Mechanizm klasy $\mathbb{DS4}$	Źródłowy język	Wykład / Materiały
Inercja, ramifikacje, niedeterminizm	AR	Wykład 2
Niewykonalność akcji	AR	Wykład 2
Semantyka kwerend (Always/Ever)	$AR / \mathcal{L}$	Wykład 3
Konflikty i dekompozycja akcji złożonych	AC	Wykład 5
Opisy częściowe i niepełna informacja	AR	Wykład 2, 7

- **Ograniczenie opisu dziedziny do akcji prostych** — w przeciwieństwie do pełnego języka AC , gdzie akcje złożone mogą być definiowane jako stałe w opisie dziedziny, $\mathbb{DS4}$ dopuszcza w specyfikacji wyłącznie akcje proste $A \in \mathcal{A}$. Akcje złożone $\mathbb{A}$ pojawiają się dopiero na poziomie języka kwerend jako zbiory akcji prostych wykonywanych symultanicznie, co wymusza dynamiczną weryfikację konfliktów zmiennych w trakcie analizy procesu.
- **Brak jawnej klauzuli *impossible* dla zbiorów akcji** — klasa $\mathbb{DS4}$ nie przewiduje bezpośredniego definiowania zakazu współbieżności dla konkretnych par akcji (tzw. *non-concurrent actions*), co jest obecne w niektórych rozszerzeniach języka AC . Wykonalność zbioru akcji jest tu determinowana przez warunek Z5 (lokalna wykonalność składowych) oraz przez mechanizm detekcji konfliktów i dekompozycji zaczerpnięty z języka AC (patrz §5, uzasadnienie w §9.2).
- **Brak mechanizmu dziedziczenia restrykcyjnego (*restricted inheritance*)** — w $\mathbb{DS4}$ nie stosuje się mechanizmów automatycznego dziedziczenia skutków akcji prostych przez akcje złożone w sposób hierarchiczny. Każda akcja złożona jest dekomponowana "od zera" do poziomu akcji prostych, a ich skutki są agregowane zgodnie z prawem inercji i warunkami integralności.

Powyższe uproszczenia nie ograniczają ogólności systemu w kontekście założeń Z1–Z7, ponieważ każdą niedozwoloną kombinację akcji można zamodelować poprzez odpowiednie zdefiniowanie warunków integralności (Z6) lub warunków wstępnych (Z4), które w przypadku konfliktu logicznego uczynią proces niewykonalnym (Z5). Mechanizm detekcji konfliktów na poziomie zmiennych w sposób naturalny zastępuje jawną definicję *impossible*, zapewniając spójność fizyczną świata dynamicznego.

2 Podstawy formalne

2.1 Sygnatura

Podstawę formalną opisu systemów w klasie $\mathbb{DS4}$ stanowi sygnatura Υ , która definiuje słownik pojęć używanych do opisu świata i zachodzących w nim zmian. Przyjmujemy definicję $\Upsilon = (\mathcal{F}, \mathcal{A})$, gdzie:

- $\mathcal{F}$ jest skończonym zbiorem **fluentów** (zmiennych zdaniowych), które opisują stan świata.
- $\mathcal{A}$ jest skończonym zbiorem **akcji prostych**.

Ważnym rozszerzeniem względem standardowej sygnatury języka AR jest fakt, że w klasie $\mathbb{DS4}$ zbiór $\mathcal{A}$ zawiera wyłącznie akcje proste – akcje złożone nie stanowią elementów sygnatury pierwotnej i są konstruowane dynamicznie w kwerendach.

2.2 Formuły zdaniowe

Język $\mathcal{L}(\Upsilon)$ umożliwia budowanie złożonych warunków (prekolekcji, skutków, więzów) za pomocą formuł zdaniowych. Gramatyka w notacji BNF dla formuł α nad sygnaturą Υ definiuje się następująco:

$$\alpha ::= f \mid \neg\alpha \mid \alpha \wedge \beta \mid \alpha \vee \beta \mid \alpha \rightarrow \beta \mid \alpha \leftrightarrow \beta \mid \top \mid \perp$$

gdzie $f \in \mathcal{F}$, a symbole $\top$ oraz $\perp$ oznaczają odpowiednio prawdę (tautologię) i fałsz (sprzeczność).

2.3 Stany

Stanem σ nazywamy funkcję wartościowania (totalną), która każdemu fluentowi ze zbioru $\mathcal{F}$ przypisuje dokładnie jedną wartość logiczną: $\sigma: \mathcal{F} \rightarrow \{0, 1\}$. Relację spełnialności formuły α w stanie σ , oznaczaną jako $\sigma \models \alpha$, definiujemy rekurencyjnie zgodnie z klasyczną logiką zdaniową:

- $\sigma \models f$ wtw, gdy $\sigma(f) = 1$,
- $\sigma \models \neg\alpha$ wtw, gdy $\sigma \not\models \alpha$,
- $\sigma \models \alpha \wedge \beta$ wtw, gdy $\sigma \models \alpha$ oraz $\sigma \models \beta$,
- $\sigma \models \alpha \vee \beta$ wtw, gdy $\sigma \models \alpha$ lub $\sigma \models \beta$,
- $\sigma \models \alpha \rightarrow \beta$ wtw, gdy $\sigma \not\models \alpha$ lub $\sigma \models \beta$.

2.4 Literały i DNF

W analizie skutków akcji i wykrywaniu konfliktów w klasie DS4 kluczowe znaczenie ma postać normalna formuł.

- **Literał** to fluent f lub jego negacja $\neg f$.
- **Dysjunkcyjna postać normalna (DNF)** to alternatywa jednej lub wielu koniunkcji literałów (składników).

Algorytm konwersji formuły α do DNF obejmuje następujące kroki: Eliminacja operatorów $\rightarrow$ oraz $\leftrightarrow$ na rzecz $\neg, \wedge, \vee$. Przesunięcie negacji do poziomu fluentów przy użyciu praw De Morgana i podwójnego przeczenia. Zastosowanie prawa dystrybucji koniunkcji względem alternatywy.

Przykład: Konwersja formuły $(\neg p \rightarrow q) \wedge r$: Eliminacja implikacji: $(\neg(\neg p) \vee q) \wedge r$. Podwójna negacja: $(p \vee q) \wedge r$. Dystrybucja: $(p \wedge r) \vee (q \wedge r)$. Wynikowa formuła jest w postaci DNF i składa się z dwóch składników: $C_1 = \{p, r\}$ oraz $C_2 = \{q, r\}$. Takie rozbiecie jest niezbędne do sprawdzania warunku rozłączności zbiorów zmiennych przy akcjach złożonych.

3 Język akcji — składnia

Projektowany język akcji służy do opisu dziedzin należących do klasy DS4 systemów dynamicznych. Zgodnie z treścią zadania, w opisach dziedzin występują wyłącznie akcje proste, tj. elementy ustalonego zbioru akcji $\mathcal{A}$. Akcje złożone, rozumiane jako zbiory akcji prostych wykonywanych współbieżnie, nie należą zatem do składni języka opisu dziedziny, lecz pojawiają się dopiero na poziomie języka kwerend, w którym budowane są procesy. Oznacza to, że na poziomie opisu dziedziny zachowujemy rdzeń języka AR, natomiast mechanizmy związane ze współbieżnością są wprowadzane dopiero przy interpretacji procesów i odpowiadających im zapytań.

Niech $\Upsilon = (\mathcal{F}, \mathcal{A})$ będzie sygnaturą, gdzie $\mathcal{F}$ jest skończonym, niepustym zbiorem fluentów, natomiast $\mathcal{A}$ jest skończonym zbiorem akcji prostych. Formuły języka budowane są nad fluentami ze zbioru $\mathcal{F}$ przy użyciu spójników klasycznego rachunku zdań. W szczególności przez α, β oraz π będziemy oznaczać dowolne formuły zdaniowe nad $\mathcal{F}$, zaś przez f — dowolny fluent należący do $\mathcal{F}$.

3.1 Zdania dziedziny — tabela zbiorcza

W projektowanym języku akcji dopuszczamy osiem podstawowych typów zdań. Pierwsze pięć z nich opisuje lokalną dynamikę lub strukturę dziedziny, natomiast trzy ostatnie pełnią rolę zdań specyfikacyjnych, odnoszących się do stanu początkowego albo do własności zachodzących po wykonaniu ciągu akcji prostych. Zestawienie wszystkich dopuszczalnych konstrukcji przedstawia tabela 2.

Tabela 2: Typy zdań języka akcji dla klasy DS4

Zdanie (zapis formalny)	Znaczenie intuicyjne	Źródło
$A \text{ causes } \alpha \text{ if } \pi$	Wykonanie akcji prostej A w stanie spełniającym π wymusza zajście efektu α .	AR
$A \text{ releases } f \text{ if } \pi$	Wykonanie akcji A w stanie spełniającym π zwalnia fluent f spod działania prawa inercji.	AR
$\text{impossible } A \text{ if } \pi$	Akcja prosta A jest niewykonalna w każdym stanie spełniającym π .	AR
$\text{always } \alpha$	Formuła α musi być prawdziwa w każdym dopuszczalnym stanie dziedziny.	AR
$\text{noninertial } f$	Fluent f nie podlega prawu inercji.	AR
$\text{initially } \alpha$	Formuła α opisuje wiedzę o stanie początkowym.	AR
$\alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu ciągu akcji prostych $A_1, \dots, A_n$ formuła α zachodzi zawsze.	AR
$\text{observable } \alpha \text{ after } A_1, \dots, A_n$	Po wykonaniu ciągu akcji prostych $A_1, \dots, A_n$ formuła α może zachodzić.	AR

W każdym z powyższych schematów zakładamy, że $A, A_1, \dots, A_n \in \mathcal{A}$, a zatem wszystkie występujące akcje są akcjami prostymi. Jest to zgodne z warunkami klasy DS4 i odróżnia poziom opisu dziedziny od poziomu języka kwerend, na którym dopiero dopuszczane są akcje złożone.

3.2 Pełna BNF dziedziny

Składnię języka akcji definiujemy następująco. Przez α, β oraz π oznaczamy dowolne formuły zdaniowe nad zbiorem fluentów $\mathcal{F}$, zaś przez A dowolną akcję prostą należącą do $\mathcal{A}$.

$$\begin{aligned}
\langle \text{dziedzina} \rangle &::= \langle \text{zdanie} \rangle \mid \langle \text{zdanie} \rangle ; \langle \text{dziedzina} \rangle \\
\langle \text{zdanie} \rangle &::= \langle \text{efekt} \rangle \mid \langle \text{zwolnienie} \rangle \mid \langle \text{niewykonalnosc} \rangle \mid \langle \text{ograniczenie} \rangle \\
&\quad \mid \langle \text{nieinercyjnosc} \rangle \mid \langle \text{pocatkowe} \rangle \mid \langle \text{wartosciowe} \rangle \mid \langle \text{obserwacyjne} \rangle \\
\langle \text{efekt} \rangle &::= A \text{ causes } \alpha \text{ if } \pi \\
\langle \text{zwolnienie} \rangle &::= A \text{ releases } f \text{ if } \pi \\
\langle \text{niewykonalnosc} \rangle &::= \text{impossible } A \text{ if } \pi \\
\langle \text{ograniczenie} \rangle &::= \text{always } \alpha \\
\langle \text{nieinercyjnosc} \rangle &::= \text{noninertial } f \\
\langle \text{pocatkowe} \rangle &::= \text{initially } \alpha \\
\langle \text{wartosciowe} \rangle &::= \alpha \text{ after } A_1, \dots, A_n \\
\langle \text{obserwacyjne} \rangle &::= \text{observable } \alpha \text{ after } A_1, \dots, A_n
\end{aligned}$$

Powyższa gramatyka obejmuje wszystkie typy zdań dopuszczalnych w języku akcji dla klasy $\mathbb{DS4}$. Warto zauważyć, że składnia nie zawiera żadnych reguł odnoszących się bezpośrednio do akcji złożonych. Ograniczenie to ma charakter zamierzony i wynika z założeń zadania: opis dziedziny dotyczy wyłącznie działań elementarnych, natomiast współbieżność pojawia się dopiero na poziomie procesów i kwerend.

3.3 Opis poszczególnych zdań

Zdanie

$$A \text{ causes } \alpha \text{ if } \pi$$

jest regułą efektową. Oznacza ono, że jeżeli akcja prosta A zostaje wykonana w stanie spełniającym warunek π , to każdy dopuszczalny stan wynikowy powinien spełniać formułę α . Reguła ta opisuje zatem bezpośredni, pewny skutek działania i stanowi podstawowy mechanizm reprezentacji zmian zachodzących pod wpływem akcji.

Zdanie

$$A \text{ releases } f \text{ if } \pi$$

jest regułą zwolnienia spod działania inercji. Jego sens polega na tym, że wykonanie akcji A w stanie spełniającym π nie wymusza zachowania poprzedniej wartości fluentu f . Fluent ten może zatem po wykonaniu akcji zachować swoją wartość albo ją zmienić. Konstrukcja ta służy do reprezentowania niedeterminizmu lokalnego i pozwala modelować skutki działań, które nie są w pełni określone.

Zdanie

$$\text{impossible } A \text{ if } \pi$$

jest regułą niewykonalności. Oznacza ono, że akcja prosta A nie może zostać wykonana w żadnym stanie spełniającym warunek π . Reguła ta nie opisuje efektu działania, lecz zakazuje samego przejścia. W szczególności podkreślamy, że w klasie $\mathbb{DS4}$ reguły tego rodzaju dotyczą wyłącznie akcji prostych; nie wprowadzamy analogicznych zdań dla akcji złożonych na poziomie opisu dziedziny.

Zdanie

$$\text{always } \alpha$$

jest ograniczeniem integralności. Oznacza ono, że formuła α musi zachodzić w każdym dopuszczalnym stanie systemu. Reguły tego typu nie odnoszą się do pojedynczego działania, lecz do

globalnej struktury dziedziny. To właśnie one umożliwiają modelowanie ramifikacji, ponieważ wymuszają spójność stanów wynikowych również w zakresie skutków ubocznych.

Zdanie

noninertial f

deklaruje, że fluent f nie podlega prawu inercji. Oznacza to, że jego wartość nie musi być zachowywana przy przejściu między stanami. Konstrukcja ta jest szczególnie użyteczna przy modelowaniu fluentów pochodnych lub zależnych od innych własności świata, których zmiana nie powinna być traktowana jako naruszenie zasady minimalnej zmiany.

Zdanie

initially α

opisuje wiedzę o stanie początkowym. W najprostszym przypadku może ono wskazywać pojedynczą własność stanu początkowego, jednak w klasie DS4 dopuszczalny jest opis częściowy. Oznacza to, że zbiór zdań tego typu nie musi wyznaczać jednego, w pełni określonego stanu, lecz może pozostawiać niektóre fluenty nieustalone.

Zdanie

α **after** $A_1, \dots, A_n$

ma charakter specyfikacyjny. Wyraża ono własność konieczną: po wykonaniu wskazanego ciągu akcji prostych $A_1, \dots, A_n$ formuła α powinna zachodzić zawsze. Zdanie to nie jest lokalną regułą przejścia dla pojedynczej akcji, lecz warunkiem odnoszącym się do zachowania modelu dla całej sekwencji działań.

Zdanie

observable α **after** $A_1, \dots, A_n$

również ma charakter specyfikacyjny. Oznacza ono, że po wykonaniu ciągu akcji prostych $A_1, \dots, A_n$ istnieje co najmniej jeden dopuszczalny przebieg prowadzący do stanu spełniającego α . Podobnie jak zdanie typu **after**, nie opisuje ono bezpośrednio lokalnej dynamiki pojedynczej akcji, lecz własność całego modelu.

3.4 Dziedzina D

Przez dziedzinę działania będziemy rozumieć skończony zbiór zdań języka akcji nad ustaloną sygnaturą $\Upsilon = (\mathcal{F}, \mathcal{A})$. Dziedzinę taką oznaczamy przez D . Zbiór D stanowi pełną formalną specyfikację systemu na poziomie akcji prostych: określa dostępne działania elementarne, ich skutki, warunki niewykonalności, ograniczenia integralności oraz wiedzę o stanie początkowym. W dalszej części opracowania to właśnie względem tak określonej dziedziny zostanie zdefiniowana semantyka akcji prostych.

4 Semantyka — poziom akcji prostej

W tej sekcji definiujemy semantykę języka akcji na poziomie akcji prostych. Niech

$$\Upsilon = (\mathcal{F}, \mathcal{A})$$

będzie ustaloną sygnaturą, gdzie $\mathcal{F}$ jest skończonym zbiorem fluentów, a $\mathcal{A}$ skończonym zbiorem akcji prostych. Niech ponadto D będzie dziedziną działania nad sygnaturą Υ . Przez stan będziemy rozumieć wartościowanie

$$\sigma : \mathcal{F} \rightarrow \{0, 1\},$$

czyli przypisanie każdemu fluentowi wartości logicznej. Dla dowolnej formuły zdaniowej α zapis

$$\sigma \models \alpha$$

oznacza, że formuła α jest prawdziwa w stanie σ .

4.1 Stany dopuszczalne Σ

Nie każdy stan jest stanem legalnym z punktu widzenia opisu dziedziny. Zdania postaci **always** narzucają globalne ograniczenia integralności, które muszą być spełnione w każdym stanie dopuszczalnym. Zbiór stanów dopuszczalnych definiujemy więc następująco:

$$\Sigma = \{\sigma : \forall(\text{always } \alpha) \in D, \sigma \models \alpha\}.$$

Zbiór Σ zawiera dokładnie te stany, które spełniają wszystkie ograniczenia integralności występujące w dziedzinie. Jeżeli pewien stan narusza choć jedno zdanie typu **always**, to nie należy do Σ i nie może wystąpić ani jako stan początkowy, ani jako wynik wykonania jakiejkolwiek akcji. W ten sposób ograniczenia integralności wyznaczają przestrzeń stanów, pomiędzy którymi mogą zachodzić przejścia. Mechanizm ten odpowiada za reprezentację ramifikacji: stan wynikowy musi być nie tylko zgodny z bezpośrednimi efektami działania, lecz również z globalną strukturą dziedziny.

4.2 Stany początkowe Σ_0

Zbiór stanów początkowych definiujemy jako podzbiór zbioru stanów dopuszczalnych spełniających wszystkie zdania początkowe:

$$\Sigma_0 = \{\sigma \in \Sigma : \forall(\text{initially } \alpha) \in D, \sigma \models \alpha\}.$$

Tak określony zbiór może być jednoelementowy albo wieloelementowy. W szczególności, jeżeli opis początkowy nie ustala wartości wszystkich fluentów, to kilka różnych stanów może być zgodnych z wiedzą początkową. Przyjęcie zbioru Σ_0 , zamiast pojedynczego wyróżnionego stanu początkowego, pozwala modelować częściowy opis stanu początkowego bez wprowadzania dodatkowych konstrukcji składniowych. Uwzględniamy jedynie te stany z Σ_0 , które są zgodne ze zdaniem wartościowymi i obserwacyjnymi występującymi w dziedzinie, tj. ze zdaniem postaci

$$\alpha \text{ after } A_1, \dots, A_n \quad \text{oraz} \quad \text{observable } \alpha \text{ after } A_1, \dots, A_n.$$

4.3 Funkcja Res_0

Niech $A \in \mathcal{A}$ będzie akcją prostą, a $\sigma \in \Sigma$ stanem dopuszczalnym. Pierwszym krokiem przy wyznaczaniu wyników wykonania akcji A w stanie σ jest określenie wszystkich stanów dopuszczalnych zgodnych z bezpośrednimi efektami aktywnych reguł efektowych. W tym celu definiujemy funkcję pomocniczą

$$\text{Res}_0(A, \sigma) = \{\sigma' \in \Sigma : \forall(A \text{ causes } \alpha \text{ if } \pi) \in D, \sigma \models \pi \Rightarrow \sigma' \models \alpha\}.$$

Zbiór $\text{Res}_0(A, \sigma)$ zawiera zatem wszystkie stany dopuszczalne, które spełniają efekty wynikające z wykonania akcji A w stanie σ . Definicja ta uwzględnia zarówno skutki bezpośrednie, jak i ograniczenia integralności, ponieważ kandydackie stany wynikowe są wybierane wyłącznie spośród elementów zbioru Σ . Funkcja Res_0 nie rozstrzyga jednak jeszcze, które z tych stanów należy uznać za rzeczywiste wyniki wykonania akcji, ponieważ na tym etapie nie jest jeszcze uwzględniona zasada inercji.

4.4 Funkcja New

W celu sformalizowania zasady minimalnej zmiany wprowadzamy funkcję

$$\text{New}(A, \sigma, \sigma'),$$

która dla akcji prostej A , stanu wejściowego σ oraz kandydata na stan wynikowy σ' wyznacza zbiór literałów nowych względem stanu wejściowego. Intuicyjnie zbiór ten opisuje zakres zmiany wymaganej do przejścia od σ do σ' .

Formalnie:

$$\text{New}(A, \sigma, \sigma') = \{ l : \sigma' \models l \text{ oraz zachodzi co najmniej jeden z warunków (1)–(2) } \},$$

gdzie:

1. $l \in \{f, \neg f\}$ dla pewnego fluentu inercyjnego $f \in \mathcal{F}_I$, oraz $\sigma(f) \neq \sigma'(f)$;
2. $l \in \{f, \neg f\}$ oraz istnieje reguła

$$(A \text{ releases } f \text{ if } \pi) \in D$$

taka, że

$$\sigma \models \pi.$$

Pierwszy warunek odpowiada rzeczywistej zmianie wartości fluentu inercyjnego. Drugi uwzględnia przypadek, w którym aktywna reguła **releases** zwalnia wskazany fluent spod działania inercji. Warto zauważyć, że fluenty zadeklarowane jako **noninertial** nie są traktowane jako zwykłe fluenty inercyjne, a zatem ich zmiana nie zwiększa kosztu przejścia w sensie zasady minimalnej zmiany.

4.5 Minimalizacja — $\text{Res}(A, \sigma)$

Właściwy zbiór wyników wykonania akcji prostej A w stanie σ otrzymujemy przez wybór tych stanów z $\text{Res}_0(A, \sigma)$, dla których zbiór $\text{New}(A, \sigma, \sigma')$ jest minimalny względem inkluzji. Definiujemy więc:

$$\text{Res}(A, \sigma) = \{ \sigma' \in \text{Res}_0(A, \sigma) : \nexists \sigma'' \in \text{Res}_0(A, \sigma) (\text{New}(A, \sigma, \sigma'') \subset \text{New}(A, \sigma, \sigma')) \}.$$

Zbiór $\text{Res}(A, \sigma)$ zawiera dokładnie te stany, które spełniają wszystkie aktywne efekty bezpośrednio i jednocześnie wprowadzają najmniejszą konieczną zmianę względem stanu wejściowego. Minimalność rozumiana jest tutaj względem inkluzji zbiorów, a nie względem ich liczności. Dzięki temu zasada inercji przyjmuje postać preferencji dla tych stanów wynikowych, które zachowują jak najwięcej własności stanu poprzedniego, o ile nie stoi to w sprzeczności z efektami akcji ani z ograniczeniami integralności.

Jeżeli kilka stanów osiąga ten sam stopień minimalności, wszystkie pozostają dopuszczalnymi wynikami wykonania akcji. Niedeterminizm może więc wynikać zarówno z obecności reguł **releases**, jak i z wielości różnych minimalnych stanów zgodnych z ograniczeniami dziedziny.

4.6 Niewykonalność

Osobnego traktowania wymaga przypadek niewykonalności akcji. Jeżeli istnieje reguła

$$(\text{impossible } A \text{ if } \pi) \in D$$

taka, że

$$\sigma \models \pi,$$

to przyjmujemy

$$\text{Res}(A, \sigma) = \emptyset.$$

Oznacza to, że akcja A nie ma w stanie σ żadnego dopuszczalnego wyniku, ponieważ nie może zostać wykonana. Warunek niewykonalności ma zatem charakter priorytetowy: jeśli akcja jest zabroniona, nie analizuje się jej efektów, lecz od razu uznaje się, że zbiór wyników jest pusty. Ujęcie to odpowiada interpretacji reguły **impossible** jako szczególnego przypadku reguły efektowej prowadzącej do sprzeczności, a zarazem zapewnia przejrzystość dalszych definicji dotyczących wykonalności procesów.

Podsumowując, semantyka akcji prostych opiera się na następującym schemacie: najpierw wyznaczany jest zbiór stanów dopuszczalnych Σ , następnie zbiór stanów początkowych Σ_0 , dalej dla każdej akcji prostej i każdego stanu dopuszczalnego budowany jest zbiór kandydatów $\text{Res}_0(A, \sigma)$, a ostatecznie spośród tych kandydatów wybierane są stany minimalne względem funkcji New. Tak zdefiniowana semantyka łączy bezpośrednie efekty działań, ograniczenia integralności, zasadę inercji, niedeterminizm oraz niewykonalność w jednolity model przejść dla akcji prostych.

5 Semantyka — akcje złożone

W niniejszym rozdziale rozszerzamy semantykę wprowadzoną w §4 na poziom akcji złożonych, rozumianych jako zbiory akcji prostych wykonywanych równocześnie. Ponieważ w klasie DS4 zdania dziedziny odnoszą się wyłącznie do akcji prostych, akcje złożone pojawiają się dopiero na poziomie procesów i są konstruowane dynamicznie. Podstawę teoretyczną stanowi język *AC*; poniższe definicje adaptują mechanizmy detekcji konfliktów i dekompozycji znane z tego języka do warunków zadania.

5.1 Akcja złożona $\mathbb{A}$

Akcją złożoną nad sygnaturą $\Upsilon = (\mathcal{F}, \mathcal{A})$ nazywamy dowolny niepusty zbiór akcji prostych

$$\mathbb{A} = \{A_1, \dots, A_k\} \subseteq \mathcal{A}, \quad \mathbb{A} \neq \emptyset.$$

Intuicyjnie, składowe akcji złożonej mają zostać wykonane równocześnie w tym samym stanie. Zbiór wszystkich akcji złożonych oznaczamy przez

$$\mathcal{A}_z = 2^{\mathcal{A}} \setminus \{\emptyset\}.$$

Zbiór ten stanowi alfabet procesów w języku kwerend (§6.1), gdzie każdy krok procesu jest elementem $\mathcal{A}_z$.

Dla wygody notacyjnej przyjmujemy konwencję, zgodnie z którą pojedyncza akcja prosta $A \in \mathcal{A}$ występująca w procesie jest skrótem oznaczającym singleton $\{A\} \in \mathcal{A}_z$. Dzięki temu semantyka akcji złożonych, wprowadzona poniżej, obejmuje jako przypadek szczególny semantykę akcji prostej z rozdziału 4.5: dla $\mathbb{A} = \{A\}$ konstrukcje z niniejszej sekcji redukują się do tych z §4.

Podkreślimy, że w klasie DS4 akcje złożone nie są elementami sygnatury ani składni opisu dziedziny. Oznacza to w szczególności, że dla akcji złożonej nie można zadeklarować reguły **causes**, **releases** ani **impossible**; jej zachowanie jest w pełni wyznaczone przez zachowanie składowych akcji prostych oraz przez mechanizm dekompozycji opisany dalej.

5.2 Konflikty

Przy równoczesnym wykonaniu kilku akcji prostych może dojść do sytuacji, w której ich skutki są ze sobą niezgodne — na przykład jedna akcja wymusza zajście literału f , a druga literału $\neg f$. W takim przypadku akcje te nazywamy *kolidującymi* w danym stanie. Definicję konfliktu przyjmujemy za językiem *AC* i formułujemy ją zawsze względem ustalonego stanu $\sigma \in \Sigma$, ponieważ zestaw aktywnych reguł **causes** oraz **releases** zależy od tego, które warunki wstępne π są w σ spełnione.

Dla danej akcji prostej $A \in \mathcal{A}$ i stanu $\sigma \in \Sigma$ przypominamy oznaczenia wprowadzone w §4:

$$E(A, \sigma) = \{\alpha : (A \text{ causes } \alpha \text{ if } \pi) \in D, \sigma \models \pi\},$$

$$R(A, \sigma) = \{f : (A \text{ releases } f \text{ if } \pi) \in D, \sigma \models \pi\}.$$

Niech $A, B \in \mathcal{A}$ będą dwiema różnymi akcjami prostymi, a $\sigma \in \Sigma$ ustalonym stanem dopuszczalnym. Mówimy, że akcje A i B **kolidują w stanie** σ , jeżeli zachodzi co najmniej jeden z dwóch warunków:

Przypadek 1 (*causes vs. causes*). Istnieją formuły $\alpha \in E(A, \sigma)$ oraz $\beta \in E(B, \sigma)$ takie, że w postaciach normalnych

$$\text{DNF}(\alpha) = C_1^\alpha \vee \dots \vee C_p^\alpha, \quad \text{DNF}(\beta) = C_1^\beta \vee \dots \vee C_q^\beta$$

występuje para składników C_i^α, C_j^β , w której pewien literal z C_i^α jest komplementarny względem pewnego literalu z C_j^β , tj. istnieje fluent $f \in \mathcal{F}$ taki, że $f \in C_i^\alpha$ i $\neg f \in C_j^\beta$, albo $\neg f \in C_i^\alpha$ i $f \in C_j^\beta$.

Przypadek 2 (*causes vs. releases*). Istnieje formuła $\alpha \in E(A, \sigma)$ oraz fluent $f \in R(B, \sigma)$ takie, że f lub $\neg f$ występuje jako literal w pewnym składniku postaci $\text{DNF}(\alpha)$. Symetrycznie dopuszczamy tę sytuację po zamianie ról A i B , tj. $\alpha \in E(B, \sigma)$ oraz $f \in R(A, \sigma)$.

Intuicyjnie, Przypadek 1 opisuje sytuację, w której obie akcje wymuszają bezpośrednio sprzeczne wartości tego samego fluentu, natomiast Przypadek 2 odnosi się do konfliktu między akcją wymuszającą wartość fluentu i akcją, która ten sam fluent jednocześnie zwalnia spod działania inercji. Posłużenie się postacią DNF jest tutaj istotne: formuły efektowe mogą być dowolnie złożonymi formułami zdaniowymi, a badanie zgodności literalów bezpośrednio w postaci DNF pozwala jednolicie rozpatrywać wszystkie alternatywy sposobu realizacji danego efektu.

Z symetrii definicji wynika, że relacja konfliktu w stanie σ jest symetryczna. Możemy zatem zakodować ją jako nieskierowany graf konfliktu

$$G_\sigma(\mathbb{A}) = (V, E), \quad V = \mathbb{A}, \quad E = \{\{A_i, A_j\} : A_i, A_j \in \mathbb{A}, A_i \neq A_j, A_i \text{ i } A_j \text{ kolidują w } \sigma\}.$$

Graf ten wykorzystamy w §5.3 do charakteryzacji dekompozycji oraz w pseudokodzie w §8.4 (algorytm 4).

5.3 Dekompozycje $\Delta(\mathbb{A}, \sigma)$

Jeżeli pewne składowe akcji złożonej $\mathbb{A}$ kolidują w stanie σ , nie możemy wykonać wszystkich z nich równocześnie. Zamiast całkowicie blokować taką akcję, za językiem AC dopuszczamy wykonanie dowolnego maksymalnego podzbioru składowych, który można bezpiecznie uruchomić razem. Taki podzbiór nazywamy dekompozycją.

Najpierw wyróżnimy akcje, które w ogóle mogą zostać wykonane pojedynczo w rozważanym stanie. Mówimy, że akcja prosta $A \in \mathcal{A}$ jest **indywidualnie wykonalna** w stanie σ wtedy i tylko wtedy, gdy $\text{Res}(A, \sigma) \neq \emptyset$. W szczególności każda akcja, dla której w stanie σ aktywne jest zdanie **impossible**, nie jest indywidualnie wykonalna (por. §4.6). Zbiór składowych indywidualnie wykonalnych oznaczamy przez

$$V_{\text{exec}}(\mathbb{A}, \sigma) = \{A \in \mathbb{A} : \text{Res}(A, \sigma) \neq \emptyset\}.$$

Definicja (dekompozycja). Podzbiór $\delta \subseteq \mathbb{A}$ jest **dekompozycją** akcji złożonej $\mathbb{A}$ w stanie σ , jeżeli spełnia trzy warunki:

- (D1) **Lokalna wykonalność.** Każda akcja $A \in \delta$ jest indywidualnie wykonalna w σ , tj. $\delta \subseteq V_{\text{exec}}(\mathbb{A}, \sigma)$.
- (D2) **Bezkonfliktowość.** Żadna para różnych akcji $A, B \in \delta$ nie koliduje w σ . Równoważnie, δ jest zbiorem niezależnym w grafie konfliktu $G_\sigma(\mathbb{A})$.
- (D3) **Maksymalność.** Nie istnieje akcja $A \in V_{\text{exec}}(\mathbb{A}, \sigma) \setminus \delta$, dla której $\delta \cup \{A\}$ nadal spełnia (D1) i (D2).

Zbiór wszystkich dekompozycji akcji $\mathbb{A}$ w stanie σ oznaczamy przez $\Delta(\mathbb{A}, \sigma)$. Maksymalność rozumiana jest względem inkluzji, a nie względem liczności, co oznacza, że zbiór $\Delta(\mathbb{A}, \sigma)$ może zawierać dekompozycje o różnej liczbie elementów.

Lemat (istnienie dekompozycji). *Jeżeli $V_{\text{exec}}(\mathbb{A}, \sigma) \neq \emptyset$, to $\Delta(\mathbb{A}, \sigma) \neq \emptyset$.*

Szkic dowodu. Niech $A_0 \in V_{\text{exec}}(\mathbb{A}, \sigma)$. Zbiór jednoelementowy $\delta_0 = \{A_0\}$ trywialnie spełnia warunek (D1) oraz, z uwagi na brak par różnych akcji, również warunek (D2). Konstruujemy ciąg zbiorów $\delta_0 \subseteq \delta_1 \subseteq \dots$ następująco: dopóki istnieje akcja $A \in V_{\text{exec}}(\mathbb{A}, \sigma) \setminus \delta_i$ taka, że $\delta_i \cup \{A\}$ wciąż spełnia (D1) oraz (D2), kładziemy $\delta_{i+1} = \delta_i \cup \{A\}$. Ponieważ $\mathbb{A}$ jest zbiorem skończonym, procedura zatrzymuje się po skończonej liczbie kroków na pewnym zbiorze δ^* . Z konstrukcji δ^* spełnia (D1) i (D2), a jego maksymalność wynika z warunku zatrzymania — żadnej akcji nie można już dodać bez naruszenia (D1) lub (D2). Zatem $\delta^* \in \Delta(\mathbb{A}, \sigma)$. $\square$

Przypadek graniczny. Jeżeli natomiast $V_{\text{exec}}(\mathbb{A}, \sigma) = \emptyset$, tj. żadna składowa akcji $\mathbb{A}$ nie jest w stanie σ indywidualnie wykonalna, to nie istnieje żadna dekompozycja spełniająca (D1), a więc

$$\Delta(\mathbb{A}, \sigma) = \emptyset \quad \text{oraz} \quad \text{Res}(\mathbb{A}, \sigma) = \emptyset.$$

Uzasadnienie takiej konwencji oraz jej interpretacja znajdują się w §9.3; tutaj traktujemy ją jako integralną część definicji.

5.4 $\text{Res}(\mathbb{A}, \sigma)$ przez dekompozycje

Mając wyznaczony zbiór dekompozycji $\Delta(\mathbb{A}, \sigma)$, definiujemy wynik wykonania akcji złożonej w trzech etapach: najpierw lokalnie, dla pojedynczej dekompozycji δ , analogicznie jak dla akcji prostej z §4, a następnie przez sumowanie wyników po wszystkich dekompozycjach.

Efekty i zwolnienia dekompozycji. Dla dekompozycji $\delta \in \Delta(\mathbb{A}, \sigma)$ definiujemy zbiorcze zestawy aktywnych efektów oraz aktywnych zwolnień:

$$E(\delta, \sigma) = \bigcup_{A \in \delta} E(A, \sigma), \quad R(\delta, \sigma) = \bigcup_{A \in \delta} R(A, \sigma).$$

Ponieważ w δ nie występują pary kolidujące, suma $E(\delta, \sigma)$ jest (jako zbiór formuł do jednoczesnego spełnienia) niesprzeczna — to właśnie gwarantuje bezkonfliktowość składowych.

Kandydaci i zbiór zmian. Definiujemy zbiór stanów dopuszczalnych zgodnych z efektami dekompozycji:

$$\text{Res}_0(\delta, \sigma) = \{\sigma' \in \Sigma : \forall \alpha \in E(\delta, \sigma), \sigma' \models \alpha\}.$$

Dla każdego kandydata $\sigma' \in \text{Res}_0(\delta, \sigma)$ wyznaczamy zbiór literałów nowych, analogicznie jak w §4.4:

$$\text{New}(\delta, \sigma, \sigma') = \{l : \sigma' \models l \text{ oraz zachodzi co najmniej jeden z warunków (1)–(2)}\},$$

gdzie (1) $l \in \{f, \neg f\}$ dla pewnego fluentu inercyjnego $f \in \mathcal{F}_I$, oraz $\sigma(f) \neq \sigma'(f)$; (2) istnieje fluent $f \in R(\delta, \sigma)$ taki, że $l \in \{f, \neg f\}$. Tak jak dla akcji prostej, fluenty zadeklarowane jako **noninertial** nie są zaliczane do zbioru zmian.

Minimalizacja lokalna. Wynikiem dekompozycji δ w stanie σ jest podzbiór kandydatów minimalnych względem zawierania zbioru New:

$$\text{Res}(\delta, \sigma) = \{\sigma' \in \text{Res}_0(\delta, \sigma) : \nexists \sigma'' \in \text{Res}_0(\delta, \sigma) (\text{New}(\delta, \sigma, \sigma'') \subset \text{New}(\delta, \sigma, \sigma'))\}.$$

Wynik akcji złożonej. Ostateczny zbiór stanów wynikowych akcji złożonej $\mathbb{A}$ w stanie σ powstaje jako suma wyników po wszystkich dekompozycjach:

$$\text{Res}(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}(\delta, \sigma). \quad (1)$$

Analogicznie rozszerzamy funkcję pomocniczą Res_0 na poziom akcji złożonej:

$$\text{Res}_0(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}_0(\delta, \sigma).$$

Dla stanu $\sigma' \in \text{Res}_0(\mathbb{A}, \sigma)$ zbiór literalów nowych na poziomie akcji złożonej definiujemy jako minimalny względem inkluzji element rodziny zbiorów $\text{New}(\delta, \sigma, \sigma')$ po tych dekompozycjach, które dopuszczają σ' jako kandydata:

$$\text{New}(\mathbb{A}, \sigma, \sigma') \in \min_{\subseteq} \{ \text{New}(\delta, \sigma, \sigma') : \delta \in \Delta(\mathbb{A}, \sigma), \sigma' \in \text{Res}_0(\delta, \sigma) \}.$$

Tak zdefiniowane rozszerzenia Res_0 oraz New są zgodne z warunkiem (M3) modelu dziedziny (patrz §6.4, w szczególności punkt (M3)): stany wynikowe akcji złożonej są to dokładnie te stany dopuszczalne, które spełniają łączne efekty jakiejś dekompozycji i wprowadzają najmniejszą konieczną zmianę.

Niedeterminizm. Istnienie wielu dekompozycji w tym samym stanie stanowi nowe, niezależne źródło niedeterminizmu w klasie $\mathbb{DS4}$ — obok niedeterminizmu pochodzącego od reguł **releases** i od wielości stanów minimalnych. W przypadku granicznym, gdy żadna para składowych nie koliduje, istnieje dokładnie jedna dekompozycja $\delta = V_{\text{exec}}(\mathbb{A}, \sigma)$, a konstrukcja (1) redukuje się do obliczenia pojedynczego wyniku analogicznego do akcji prostej.

5.5 Uwaga o „restricted inheritance”

W pełnym języku AC dziedzina dopuszcza zdania dotyczące bezpośrednio akcji złożonych, na przykład zdania typu $\mathbb{A}$ **causes** α **if** π dla $\mathbb{A}$ będącego zbiorem akcji. W takim przypadku efekt ogłoszony dla $\mathbb{A}$ nie musi być w ogólności dziedziczony przez wszystkie jej podzbiory — mechanizm ten nosi w literaturze nazwę *restricted inheritance*.

W klasie $\mathbb{DS4}$ opisy dziedziny zawierają wyłącznie zdania odnoszące się do akcji prostych (patrz §3); żadne zdanie nie deklaruje efektu zbioru akcji. Z tego powodu problem ograniczonego dziedziczenia nie występuje: efekty każdej akcji prostej A_i są dziedziczone w całości przez każdą dekompozycję zawierającą A_i , a łączny efekt dekompozycji powstaje jako suma zbiorów $E(A_i, \sigma)$ oraz $R(A_i, \sigma)$ zgodnie z §5.4. Stanowi to świadome uproszczenie mechanizmu AC dopuszczalne w zakresie zadania projektowego.

5.6 Porównanie z treścią projektu

Treść zadania projektowego proponuje prostszy mechanizm obsługi współbieżności: akcje złożone wykonują się poprawnie, jeżeli ich składowe operują na rozłącznych zbiorach fluentów. Poniżej pokazujemy, że nasz mechanizm detekcji konfliktów zaczerpnięty z AC jest formalnie silniejszy — warunek rozłączności jest jego szczególnym przypadkiem. Merytoryczne uzasadnienie wyboru ogólniejszego mechanizmu znajduje się w §9.2; tutaj ograniczamy się do argumentu formalnego.

Dla akcji prostej $A \in \mathcal{A}$ oraz stanu $\sigma \in \Sigma$ definiujemy zbiór fluentów, na które A *potencjalnie wpływa* w stanie σ :

$$\text{Vars}(A, \sigma) = \left(\bigcup_{\alpha \in E(A, \sigma)} \text{fl}(\alpha) \right) \cup R(A, \sigma),$$

gdzie $\text{fl}(\alpha)$ oznacza zbiór fluentów występujących w formule α (niezależnie od polaryzacji literalu). Zbiór $\text{Vars}(A, \sigma)$ obejmuje zatem wszystkie fluenty, które mogą zmienić swoją wartość pod wpływem aktywnych reguł **causes** lub **releases** dla akcji A .

Twierdzenie (rozłączność implikuje brak konfliktu). *Niech $A, B \in \mathcal{A}$ będą różnymi akcjami prostymi i niech $\sigma \in \Sigma$ będzie dowolnym stanem dopuszczalnym. Jeżeli $\text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma) = \emptyset$, to akcje A i B nie kolidują w σ w sensie §5.2.*

Dowód. Załóżmy nie wprost, że akcje A i B kolidują w σ . Rozważymy oba przypadki definicji konfliktu.

Przypadek 1. Istnieją $\alpha \in E(A, \sigma)$, $\beta \in E(B, \sigma)$ oraz fluent f taki, że f lub $\neg f$ występuje w pewnym składniku $\text{DNF}(\alpha)$, a komplementarny literal — w pewnym składniku $\text{DNF}(\beta)$. W szczególności fluent f pojawia się w formule α oraz w formule β , zatem $f \in \text{fl}(\alpha) \subseteq \text{Vars}(A, \sigma)$ oraz $f \in \text{fl}(\beta) \subseteq \text{Vars}(B, \sigma)$. Stąd $f \in \text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma)$ — sprzeczność z założeniem rozłączności.

Przypadek 2. Istnieje (bez utraty ogólności) formuła $\alpha \in E(A, \sigma)$ oraz fluent $f \in R(B, \sigma)$ taki, że f lub $\neg f$ występuje w pewnym składniku $\text{DNF}(\alpha)$. Wówczas $f \in \text{fl}(\alpha) \subseteq \text{Vars}(A, \sigma)$ oraz $f \in R(B, \sigma) \subseteq \text{Vars}(B, \sigma)$, co daje $f \in \text{Vars}(A, \sigma) \cap \text{Vars}(B, \sigma)$ — ponownie sprzeczność. W obu przypadkach otrzymaliśmy sprzeczność, więc akcje A i B nie kolidują w σ . $\square$

Wniosek. Jeżeli wszystkie pary składowych akcji złożonej $\mathbb{A}$ operują na parami rozłącznych zbiorach fluentów, to graf konfliktu $G_\sigma(\mathbb{A})$ nie zawiera żadnej krawędzi, a zatem $\Delta(\mathbb{A}, \sigma) = \{V_{\text{exec}}(\mathbb{A}, \sigma)\}$. W tej sytuacji mechanizm AC i warunek rozłączności z treści zadania prowadzą do identycznych wyników. Przykład §10.2 (producent i konsumenci) pokazuje natomiast, że odwrotna implikacja nie zachodzi: para akcji operująca na częściowo wspólnym zbiorze fluentów może pozostać bezkonfliktowa, o ile nie prowadzi do komplementarnych literalów w sensie Przypadku 1 ani do sytuacji z Przypadku 2.

6 Procesy i model dziedziny

6.1 Proces $\mathbb{P}$

Proces $\mathbb{P} = (\mathbb{A}_1, \dots, \mathbb{A}_n)$, $n > 0$ to ciąg akcji złożonych wykonywanych jedna po drugiej. Proces modeluje uporządkowany zbiór akcji następujących po sobie. Każda z nich może być akcją złożoną, która z kolei przedstawia zdarzenia równoległe.

Przez $\mathcal{P} = \mathcal{A}_z^*$ oznaczamy zbiór wszystkich procesów.

6.2 Funkcja Ψ

Funkcja częściowa $\Psi : \mathcal{P} \times \Sigma \rightarrow \Sigma$ podaje możliwy stan osiągnięty po wykonaniu procesu $\mathbb{P} = (\mathbb{A}_1, \dots, \mathbb{A}_k)$ ze stanu σ . Definiujemy ją rekurencyjnie.

Dla pustego procesu:

$$\Psi(\varepsilon, \sigma) = \{\sigma\}$$

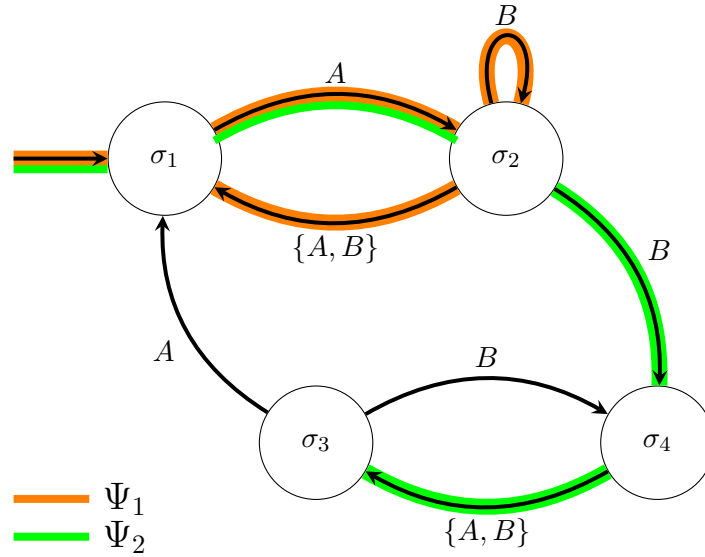
Jeśli $\Psi((\mathbb{A}_1, \dots, \mathbb{A}_k), \sigma)$ jest zdefiniowane to zachodzi:

$$\Psi((\mathbb{A}_1, \dots, \mathbb{A}_k), \sigma) \in \text{Res}((\mathbb{A}_1, \dots, \mathbb{A}_{k-1}), \sigma)$$

Jedna instancja funkcji Ψ przedstawia ślad wykonania procesu P , wraz ze stanami pośrednimi. Obrazuje to rysunek 1.

6.3 Prawdziwość zdań wartościowych

Zdania wartościowe są prawdziwe w $S = (\Sigma, \Sigma_0, \text{Res})$ jeśli zachodzą warunki:



Rysunek 1: Przykładowe funkcje Ψ dla procesu $\mathbb{P} = (A, B, \{A, B\})$

α **after** $A_1, A_2, \dots, A_n$

Dla każdego $\sigma_0 \in \Sigma_0$ i dla każdej funkcji Ψ_S , jeżeli $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0)$ jest zdefiniowane, to $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0) \models \alpha$.

observable α **after** $A_1, A_2, \dots, A_n$

Istnieje $\sigma_0 \in \Sigma_0$ oraz funkcja Ψ_S taka, że $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0)$ jest zdefiniowane oraz $\Psi_S((A_1, A_2, \dots, A_n), \sigma_0) \models \alpha$.

6.4 Model dziedziny

Struktura $S = (\Sigma, \Sigma_0, \text{Res})$ jest modelem dziedziny D , jeśli:

- (M1) Σ jest zbiorem wszystkich stanów w D , tj. zbiorem wszystkich stanów dopuszczalnych (patrz 4.1)
- (M2) Każde zdanie wartościowe i obserwacyjne (**initially**, **after** i **observable ... after**) z D jest prawdziwe w S (patrz 6.3, 4.2)
- (M3) Dla każdej akcji $\mathbb{A} \subseteq \mathcal{A}$ i dla każdego stanu $\sigma \in \Sigma$ zbiór $\text{Res}(\mathbb{A}, \sigma) \subseteq \text{Res}_0(\mathbb{A}, \sigma)$ jest zbiorem wszystkich stanów $\sigma' \in \text{Res}_0(\mathbb{A}, \sigma)$, takich że zbiór $\text{New}(\mathbb{A}, \sigma, \sigma')$ jest minimalny ze względu na zawieranie zbiorów (patrz 4.5).

Warunek **M1** wymusza zachowanie zdań integralnościowych **always**. Warunek **M2** zachowanie zdań wartościowych. A warunek **M3** zachowanie własności inercji, tj. fluenty niewspomniane w akcjach zachowują swoją wartość.

Definicja modelu odpowiada językowi AC.

6.5 Jednoznaczność modelu

Model S spełniający powyższe warunki jest jedyny dla dziedziny D . Wynika to z faktu, że wszystkie trzy elementy modelu są wyznaczone w sposób jednoznaczny.

Σ determinowana jest w całości przez zdania **always**. Σ_0 determinowana jest przez zdania **initially**, **after** i **observable ... after**. Res powstaje na podstawie zdań **causes**, **releases** i **impossible**. Specyfika języka wymaga użycia niedeterminizmu – akcja może przeprowadzać

stan do kilku różnych stanów. W takich sytuacjach Res to zawsze zbiór wszystkich możliwych stanów docelowych, więc jest jedyny.

Konsekwencją jednoznaczności modelu jest to, że kwantyfikator po modelach występujący w definicji relacji $D \approx Q$ (§7.3) sprowadza się do sprawdzenia w tym jednym modelu.

7 Język kwerend

7.1 Zdania języka kwerend

Język kwerend zawiera cztery możliwe zapytania. Dwie pierwsze dotyczą osiągania stanu po wykonaniu procesu. Dwie następne odpowiadają na pytanie, czy proces jest wykonalny. W obu przypadkach kwerendy występują w wersjach *necessary* i *possibly*: wersja *necessary* wymaga, by warunek zachodził dla *każdego* stanu początkowego $\sigma_0 \in \Sigma_0$ oraz *każdego* śladu wykonania Ψ , natomiast wersja *possibly* wymaga, by warunek zachodził dla *pewnego* stanu początkowego i *pewnego* śladu wykonania. Zestawienie wszystkich dopuszczalnych konstrukcji przedstawia tabela 3.

Tabela 3: Typy zdań języka kwerend dla klasy DS4

Zdanie (zapis formalny)	Znaczenie intuicyjne
necessary α after $\mathbb{P}$	Czy po wykonaniu procesu $\mathbb{P}$ z każdego stanu początkowego, na każdym śladzie wykonania, formuła α zachodzi?
possibly α after $\mathbb{P}$	Czy istnieje stan początkowy i ślad wykonania procesu $\mathbb{P}$ taki, że po jego wykonaniu formuła α zachodzi?
necessary executable $\mathbb{P}$	Czy wykonanie procesu $\mathbb{P}$ jest możliwe z każdego stanu początkowego na każdym śladzie wykonania?
possibly executable $\mathbb{P}$	Czy istnieje stan początkowy i ślad wykonania, na którym proces $\mathbb{P}$ jest wykonalny?

7.2 Składnia

W języku kwerend występują procesy, będącymi ciągiem akcji złożonych. Składnię dla tych struktur definiujemy następująco:

$$\begin{aligned}
\langle \text{proces} \rangle &::= "(" \langle \text{proces-wnetrze} \rangle ")" \\
\langle \text{proces-wnetrze} \rangle &::= \langle \text{proces-wnetrze} \rangle ", " \langle \text{akcja-zlozona} \rangle | \langle \text{akcja-zlozona} \rangle \\
\langle \text{akcja-zlozona} \rangle &::= "{" \langle \text{akcja-zlozona-wnetrze} \rangle "}" \\
\langle \text{akcja-zlozona-wnetrze} \rangle &::= \langle \text{akcja-zlozona-wnetrze} \rangle ", " \langle \text{akcja} \rangle | \langle \text{akcja} \rangle
\end{aligned}$$

Właściwą składnię języka kwerend definiujemy następująco.

$$\begin{aligned}
\langle kw\text{-wartosciowa-nec} \rangle &::= \text{"necessary"} \langle formula \rangle \text{"after"} \langle proces \rangle \\
\langle kw\text{-wartosciowa-pos} \rangle &::= \text{"possibly"} \langle formula \rangle \text{"after"} \langle proces \rangle \\
\langle kw\text{-wykonalnosci-nec} \rangle &::= \text{"necessary executable"} \langle proces \rangle \\
\langle kw\text{-wykonalnosci-pos} \rangle &::= \text{"possibly executable"} \langle proces \rangle
\end{aligned}$$

7.3 Semantyka formalna

Relacja $D \models Q$

Kwerenda Q jest konsekwencją dziedziny D , gdy zachodzi w każdym modelu S . Z uwagi, że w naszym przypadku model dziedziny jest tylko jeden (patrz 6.5), wystarczy że Q zachodzi w dowolnym modelu.

Kwerendy dla wielu stanów początkowych

Z założeń problemu model może mieć wiele różnych stanów początkowych $\sigma_0 \in \Sigma_0$, a z powodu niedeterminizmu Res każdy taki stan może generować wiele różnych śladów wykonania Ψ_S . Kwerendę **necessary** uznajemy za prawdziwą, gdy warunek kwerendy jest spełniony dla *każdego* stanu początkowego oraz *każdego* śladu wykonania. Kwerendę **possibly** uznajemy za prawdziwą, gdy warunek kwerendy jest spełniony dla *pewnego* stanu początkowego i *pewnego* śladu wykonania.

Definicja spełnialności kwerend

$Q : \text{necessary } \alpha \text{ after } \mathbb{P}$

$D \models Q$ wtw gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu początkowego $\sigma_0 \in \Sigma_0$ i dla każdej funkcji Ψ_S wartość $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana oraz zachodzi $\Psi_S(\mathbb{P}, \sigma_0) \models \alpha$.

$Q : \text{possibly } \alpha \text{ after } \mathbb{P}$

$D \models Q$ wtw gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ istnieje stan początkowy $\sigma_0 \in \Sigma_0$ oraz funkcja Ψ_S takie, że $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowane oraz zachodzi $\Psi_S(\mathbb{P}, \sigma_0) \models \alpha$.

$Q : \text{necessary executable } \mathbb{P}$

$D \models Q$ wtw gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ dla każdego stanu początkowego $\sigma_0 \in \Sigma_0$ i dla każdej funkcji Ψ_S wartość $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowana.

$Q : \text{possibly executable } \mathbb{P}$

$D \models Q$ wtw gdy w modelu $S = (\Sigma, \Sigma_0, \text{Res})$ istnieje stan początkowy $\sigma_0 \in \Sigma_0$ oraz funkcja Ψ_S takie, że $\Psi_S(\mathbb{P}, \sigma_0)$ jest zdefiniowane.

8 Algorytmizacja (pseudokod)

W tej sekcji pokazujemy, w jaki sposób definicje semantyczne z poprzednich rozdziałów można przełożyć na konkretne procedury obliczeniowe. Przyjmujemy podejście jawne, oparte na pełnej eksploracji przestrzeni stanów oraz wszystkich istotnych kombinacji akcji. Nie jest to rozwiązanie asymptotycznie optymalne, ale ma dwie ważne zalety. Po pierwsze, bardzo dobrze odpowiada przyjętej semantyce formalnej. Po drugie, dla rozmiarów przykładów rozważanych w projekcie pozostaje całkowicie wykonalne obliczeniowo.

W dalszej części zakładamy, że formuły zdaniowe są reprezentowane w postaci drzew składowych, dzięki czemu sprawdzenie, czy stan σ spełnia formułę α , może być realizowane przez standardową rekurencyjną procedurę ewaluacji. Zakładamy również istnienie pomocniczej procedury przekształcania formuły do postaci DNF, potrzebnej przy wykrywaniu konfliktów.

8.1 Generacja Σ

Pierwszym krokiem jest wyznaczenie zbioru stanów dopuszczalnych Σ . Zgodnie z definicją z rozdziału 4 stan dopuszczalny to takie wartościowanie fluentów, które spełnia wszystkie ograniczenia integralności postaci **always**. Ponieważ każdy fluent przyjmuje jedną z dwóch wartości logicznych, dla zbioru fluentów $\mathcal{F}$ istnieje dokładnie

$$2^{|\mathcal{F}|}$$

możliwych wartościowań. Algorytm polega więc na przejściu po wszystkich tych wartościowaniach i odrzuceniu tych, które naruszają choć jedno ograniczenie integralności.

Formalnie:

$$\Sigma = \{\sigma : \forall(\mathbf{always} \alpha) \in D, \sigma \models \alpha\}.$$

Po wygenerowaniu Σ w analogiczny sposób wyznaczamy zbiór stanów początkowych:

$$\Sigma_0 = \{\sigma \in \Sigma : \forall(\mathbf{initially} \alpha) \in D, \sigma \models \alpha\}.$$

Warto podkreślić, że w klasie DS4 zbiór Σ_0 może zawierać więcej niż jeden stan, ponieważ opis początkowy może być częściowy. W konsekwencji dalsze algorytmy muszą brać pod uwagę wszystkie stany początkowe zgodne z dziedziną.

Pseudokod dla algorytmu przedstawiono w algorytmie 1.

Algorithm 1 Pseudokod dla algorytmu generującego zbiór stanów dopuszczalnych

```

1: procedure GENERUJSIGMA( $\mathcal{F}, D$ )
2:    $\Sigma \leftarrow$  wszystkie wartościowania  $\sigma: \mathcal{F} \rightarrow \{0, 1\}$ 
3:   for all  $\sigma \in \Sigma$  do
4:     for all zdanie (always  $\alpha$ )  $\in D$  do
5:       if  $\sigma \not\models \alpha$  then
6:         usuń  $\sigma$  z  $\Sigma$ 
7:   return  $\Sigma$ 

```

Uwagi o złożoności. Jeżeli $m = |\mathcal{F}|$, to liczba wszystkich kandydatów na stan wynosi 2^m . Otrzymujemy zatem koszt rzędu

$$O(2^{|\mathcal{F}|} \cdot c_{\mathbf{always}})$$

dla wyznaczenia Σ , gdzie $c_{\mathbf{always}}$ oznacza koszt sprawdzenia wszystkich zdań integralnościowych. Wyznaczenie Σ_0 ma koszt

$$O(|\Sigma| \cdot c_{\mathbf{initially}}).$$

8.2 Res dla akcji prostej

Dla akcji prostej $A \in \mathcal{A}$ oraz stanu $\sigma \in \Sigma$ chcemy obliczyć zbiór wszystkich możliwych stanów wynikowych $\text{Res}(A, \sigma)$. Procedura ta jest bezpośrednią implementacją definicji z rozdziału 4 i przebiega w trzech etapach: najpierw sprawdzana jest niewykonalność, następnie budowany jest zbiór kandydatów $\text{Res}_0(A, \sigma)$, a na końcu wykonywana jest minimalizacja względem funkcji New.

W pierwszym kroku sprawdzamy, czy akcja jest w stanie σ niewykonalna. Jeżeli istnieje reguła

$$(\mathbf{impossible} \ A \ \text{if} \ \pi) \in D$$

taka, że

$$\sigma \models \pi,$$

to zgodnie z przyjętą semantyką przyjmujemy od razu:

$$\text{Res}(A, \sigma) = \emptyset.$$

Jeżeli akcja nie jest niewykonalna, wyznaczamy zbiór wszystkich aktywnych efektów bezpośrednich oraz zbiór aktywnych zwolnień spod inercji:

$$E(A, \sigma) = \{\alpha : (A \text{ **causes** } \alpha \text{ **if** } \pi) \in D \text{ oraz } \sigma \models \pi\},$$

$$R(A, \sigma) = \{f : (A \text{ **releases** } f \text{ **if** } \pi) \in D \text{ oraz } \sigma \models \pi\}.$$

Następnie przeglądamy wszystkie stany dopuszczalne $\sigma' \in \Sigma$ i pozostawiamy tylko te, które spełniają wszystkie aktywne efekty z $E(A, \sigma)$. W ten sposób otrzymujemy zbiór kandydatów:

$$\text{Res}_0(A, \sigma) = \{\sigma' \in \Sigma : \forall \alpha \in E(A, \sigma), \sigma' \models \alpha\}.$$

Dla każdego kandydata $\sigma' \in \text{Res}_0(A, \sigma)$ obliczamy następnie zbiór

$$\text{New}(A, \sigma, \sigma'),$$

czyli zbiór literałów nowych względem stanu wejściowego. Zgodnie z definicją z rozdziału 4 do zbioru tego trafiają:

1. literały odpowiadające fluentom inercyjnym, które rzeczywiście zmieniły wartość,
2. literały odpowiadające fluentom zwolnionym spod inercji przez aktywne reguły **releases**.

Na końcu z $\text{Res}_0(A, \sigma)$ zostawiamy tylko te stany, dla których zbiór New jest minimalny względem zawierania:

$$\text{Res}(A, \sigma) = \{\sigma' \in \text{Res}_0(A, \sigma) : \nexists \sigma'' \in \text{Res}_0(A, \sigma) (\text{New}(A, \sigma, \sigma'') \subset \text{New}(A, \sigma, \sigma'))\}.$$

Pseudokod dla algorytmu przedstawiono w algorytmie 2.

Algorithm 2 Pseudokod dla algorytmu wyznaczający funkcję Res

```

1: procedure GENERUJRES0( $A, \sigma, E$ )
2:    $\text{Res}_0 \leftarrow \emptyset$ 
3:   for all  $\sigma' \in \Sigma$  do
4:     if  $\sigma' \models \alpha$  dla każdego  $\alpha \in E$  then
5:       dodaj  $\sigma'$  do  $\text{Res}_0$ 
6:   return  $\text{Res}_0$ 
7: procedure GENERUJNEW( $A, \sigma, \sigma', R$ )
8:    $\text{New} \leftarrow \emptyset$ 
9:   for all  $f \in \mathcal{F}$  do
10:    if  $(\text{noninertial } f) \in D$  then
11:      continue
12:     $l \leftarrow$  ten spośród literałów  $\{f, \neg f\}$ , który spełnia  $\sigma' \models l$ 
13:    if  $\sigma(f) \neq \sigma'(f)$  lub  $f \in R$  then
14:      dodaj  $l$  do  $\text{New}$ 
15:   return  $\text{New}$ 
16: procedure GENERUJRES( $A, \sigma$ )
17:   for all  $(\text{impossible } A \text{ if } \pi) \in D$  do
18:     if  $\sigma \models \pi$  then
19:       return  $\emptyset$ 
20:    $E \leftarrow \emptyset, R \leftarrow \emptyset$ 
21:   for all  $(A \text{ causes } \alpha \text{ if } \pi) \in D$  do
22:     if  $\sigma \models \pi$  then
23:       dodaj  $\alpha$  do  $E$ 
24:   for all  $(A \text{ releases } f \text{ if } \pi) \in D$  do
25:     if  $\sigma \models \pi$  then
26:       dodaj  $f$  do  $R$ 
27:    $\text{Res}_0 \leftarrow \text{GENERUJRES0}(A, \sigma, E)$ 
28:   for all  $\sigma' \in \text{Res}_0$  do
29:      $\mathcal{N}[\sigma'] \leftarrow \text{GENERUJNEW}(A, \sigma, \sigma', R)$ 
30:    $\text{Res} \leftarrow \text{Res}_0$ 
31:   for all  $\sigma' \in \text{Res}_0$  do
32:     if istnieje  $\sigma'' \in \text{Res}_0$  takie, że  $\mathcal{N}[\sigma''] \subsetneq \mathcal{N}[\sigma']$  then
33:       usuń  $\sigma'$  z  $\text{Res}$ 
34:   return  $\text{Res}$ 

```

Uwagi implementacyjne. Minimalizacja nie odbywa się względem liczności zbioru New, lecz względem relacji zawierania. Oznacza to, że dwa różne stany wynikowe mogą pozostać równocześnie dopuszczalne, jeżeli odpowiadające im zbiory zmian są nieporównywalne.

Uwagi o złożoności. Jeżeli oznaczymy przez $N = |\Sigma|$ liczbę stanów dopuszczalnych, to koszt wyznaczenia $\text{Res}_0(A, \sigma)$ jest liniowy względem N , natomiast końcowa minimalizacja wymaga w najprostszym wariancie porównania kandydatów parami. W rezultacie naiwny koszt obliczenia $\text{Res}(A, \sigma)$ jest rzędu

$$O(N \cdot c_{\text{efekty}} + |\text{Res}_0(A, \sigma)|^2 \cdot |\mathcal{F}|).$$

8.3 Generacja Σ_0

Wyznaczanie stanów początkowych Σ_0 opiera się na analizie zdań wartościowych (**initially**, **after** i **observable ... after**). Dla każdego takiego zdania znajdujemy stany σ z Σ , które spełniają

warunek α . Następnie przechodzimy przez akcję w odwrotnej kolejności. Stany, przez które przechodzimy są zdefiniowane jako odwrotność funkcji Res.

Schemat działania przedstawiona na algorytmie 3.

Zdania typu **initially** traktujemy w tym algorytmie jak zdania typu **after** z zerową długością listy akcji.

Algorithm 3 Pseudokod dla algorytmu wyznaczającego stany początkowe

```

1: procedure GENERUJSIGMA0ZDANIE(Res,  $\Sigma$ , observable  $\alpha$ ,  $(A_1, A_2, \dots, A_n)$ )
2:    $\Sigma_0^{(n)} \leftarrow \{\sigma \in \Sigma : \sigma \models \alpha\}$ 
3:   for all  $A_i \in (A_n, A_{n-1}, \dots, A_1)$  do
4:      $\Sigma_{A,\sigma} \leftarrow \bigcup_{\sigma \in \Sigma} \text{Res}(A, \sigma)$ 
5:     if observable then
6:       if  $\Sigma_0^{(i)} \subseteq \Sigma_{A,\sigma}$  then
7:          $\Sigma_0^{(i-1)} \leftarrow \Sigma_0^{(i)} \cap \Sigma_{A,\sigma}$ 
8:       else
9:         if  $\Sigma_0^{(i)} = \Sigma_{A,\sigma}$  then
10:           $\Sigma_0^{(i-1)} \leftarrow \Sigma_{A,\sigma}$ 
11:     return  $\Sigma_0^{(0)}$ 
12: procedure GENERUJSIGMA0(Res,  $\Sigma$ )
13:    $\Sigma_0 \leftarrow \Sigma$ 
14:   for all  $((\text{observable}) \alpha \text{ after } A_1, A_2, \dots, A_n) \in D$  do
15:      $\Sigma'_0 \leftarrow \text{GenerujSigma0Zdanie}(\text{Res}, \Sigma, \text{observable}\alpha, (A_1, A_2, \dots, A_n))$ 
16:      $\Sigma_0 \leftarrow \Sigma_0 \cap \Sigma'_0$ 
17:   return  $\Sigma_0$ 

```

8.4 Wykrywanie konfliktów

Dla akcji złożonej

$$\mathbb{A} = \{A_1, \dots, A_k\}$$

konieczne jest wykrycie konfliktów pomiędzy parami akcji prostych. W tym miejscu korzystamy z idei wprowadzonych dla języka AC, ale w wersji dostosowanej do klasy $\mathbb{DS}4$. Ponieważ w naszej dziedzinie występują wyłącznie akcje proste, konflikty badamy między parami akcji prostych należących do $\mathbb{A}$.

Zgodnie z definicją z rozdziału 5 konflikt zawsze badamy względem konkretnego stanu σ , ponieważ to od niego zależy, które reguły **causes** oraz **releases** są aktywne.

Dla każdej pary różnych akcji $A_i, A_j \in \mathbb{A}$ wykonujemy następujące kroki:

1. wyznaczamy aktywne efekty obu akcji w stanie σ ,
2. sprowadzamy te efekty do postaci DNF,
3. sprawdzamy, czy zachodzi któryś z dwóch przypadków konfliktu:
 - (a) **causes–causes**: istnieją składniki DNF zawierające literały komplementarne,
 - (b) **causes–releases**: fluent zwolniony przez jedną akcję występuje w składniku DNF efektu drugiej akcji.

Ponieważ relacja konfliktu jest symetryczna, wynik tej procedury można reprezentować przez nieskierowany graf konfliktu

$$G_\sigma(\mathbb{A}) = (V, E),$$

gdzie wierzchołkami są akcje proste należące do $\mathbb{A}$, a krawędź między A_i i A_j oznacza, że akcje te są w konflikcie w stanie σ .

Pseudokod dla algorytmu przedstawiono w algorytmie 4.

Algorithm 4 Pseudokod dla algorytmu wykrywającego konflikty

```

1: procedure WYKRYJKONFLIKTY( $\mathbb{A}, \sigma, D$ )
2:    $E_G \leftarrow \emptyset$ 
3:   for all para różnych akcji  $A_i, A_j \in \mathbb{A}$  do
4:     wyznacz aktywne efekty i aktywne zwolnienia obu akcji w stanie  $\sigma$ 
5:     zamień aktywne efekty obu akcji na DNF
6:     if zachodzi konflikt typu causes-causes or konflikt typu causes-releases then
7:       dodaj krawędź  $\{A_i, A_j\}$  do  $E_G$ 
   return  $G_\sigma(\mathbb{A}) = (\mathbb{A}, E_G)$ 

```

Uwagi o złożoności. Dla $k = |\mathbb{A}|$ liczba par akcji jest rzędu $O(k^2)$. Główny koszt pojedynczego porównania zależy od złożoności konwersji formuł do DNF oraz od liczby składników w tej postaci. W naszych przykładach formuły efektów są krótkie, dlatego koszt ten pozostaje niewielki.

8.5 Generacja dekompozycji

Po zbudowaniu grafu konfliktu wyznaczamy dekompozycje akcji złożonej. Korzystamy tutaj z definicji przyjętej w rozdziale 5 dla klasy $\mathbb{DS4}$. Oznacza to, że dekompozycja $\delta \in \Delta(\mathbb{A}, \sigma)$ jest maksymalnym podzbiorem akcji prostych należących do $\mathbb{A}$, który:

1. składa się wyłącznie z akcji indywidualnie wykonalnych w stanie σ ,
2. nie zawiera par akcji pozostających ze sobą w konflikcie,
3. jest maksymalny względem inkluzji.

Najprostsza implementacja polega na enumeracji wszystkich podzbiorów akcji. Najpierw odrzucamy akcje indywidualnie niewykonalne, a następnie spośród pozostałych podzbiorów wybieramy te, które są bezkonfliktowe i maksymalne.

Niech

$$V_{\text{exec}} = \{A \in \mathbb{A} : \text{Res}(A, \sigma) \neq \emptyset\}$$

oznacza zbiór akcji prostych wykonalnych indywidualnie w stanie σ . Na zbiorze tym budujemy graf konfliktu $G_\sigma(\mathbb{A})$. Dekompozycje są wtedy dokładnie maksymalnymi bezkonfliktowymi podzbiórmi zbioru V_{exec} .

Pseudokod dla algorytmu przedstawiono w algorytmie 5.

Algorithm 5 Pseudokod dla algorytmu generującego dekompozycje

```
1: procedure GENERUJDEKOMPOZYCJE( $\mathbb{A}, \sigma, D$ )
2:    $V_{\text{exec}} \leftarrow \emptyset$ 
3:   for all  $A \in \mathbb{A}$  do
4:     if  $\text{Res}(A, \sigma) \neq \emptyset$  then
5:       dodaj  $A$  do  $V_{\text{exec}}$ 
6:   if  $V_{\text{exec}} = \emptyset$  then return  $\emptyset$ 
7:   zbuduj graf konfliktu  $G_\sigma(\mathbb{A})$  na zbiorze  $V_{\text{exec}}$ 
8:    $\mathcal{C} \leftarrow \emptyset$ 
9:   for all podzbiór  $X \subseteq V_{\text{exec}}$  do
10:    if żadna para akcji z  $X$  nie jest połączona krawędzią konfliktu then
11:      dodaj  $X$  do  $\mathcal{C}$ 
12:   usuń z  $\mathcal{C}$  wszystkie zbiory, które nie są maksymalne względem inkluzji
   return  $\Delta(\mathbb{A}, \sigma) = \mathcal{C}$ 
```

Jeżeli zbiór V_{exec} jest pusty, to nie istnieje żadna dekompozycja, a więc przyjmujemy

$$\Delta(\mathbb{A}, \sigma) = \emptyset \quad \text{oraz} \quad \text{Res}(\mathbb{A}, \sigma) = \emptyset.$$

Uwagi o złożoności. Dla $k = |\mathbb{A}|$ liczba wszystkich podzbiorów wynosi 2^k . Sprawdzenie bezkonfliktowości pojedynczego podzbioru można wykonać w czasie $O(k^2)$, stąd prosty koszt rzędu

$$O(2^{|\mathbb{A}|} \cdot |\mathbb{A}|^2).$$

Jest to koszt wykładniczy, ale dla niewielkich przykładów projektowych pozostaje akceptowalny.

8.6 Res dla akcji złożonej

Po wyznaczeniu zbioru dekompozycji $\Delta(\mathbb{A}, \sigma)$ możemy obliczyć wynik akcji złożonej. W tym miejscu również stosujemy adaptację idei z języka AC do klasy $\mathbb{DS4}$. Ponieważ w opisie dziedziny występują wyłącznie akcje proste, efekty danej dekompozycji otrzymujemy przez połączenie aktywnych efektów i aktywnych zwolnień wszystkich jej składowych.

Dla każdej dekompozycji

$$\delta = \{A_{i_1}, \dots, A_{i_r}\} \in \Delta(\mathbb{A}, \sigma)$$

definiujemy:

$$E(\delta, \sigma) = \bigcup_{A \in \delta} E(A, \sigma), \quad R(\delta, \sigma) = \bigcup_{A \in \delta} R(A, \sigma).$$

Następnie:

$$\text{Res}_0(\delta, \sigma) = \{\sigma' \in \Sigma : \forall \alpha \in E(\delta, \sigma), \sigma' \models \alpha\}.$$

Dla każdego $\sigma' \in \text{Res}_0(\delta, \sigma)$ obliczamy zbiór

$$\text{New}(\delta, \sigma, \sigma')$$

analogicznie jak dla akcji prostej, z tą różnicą, że uwzględniamy łącznie wszystkie zwolnienia wynikające z akcji należących do δ . Następnie zachowujemy wyłącznie stany minimalne:

$$\text{Res}(\delta, \sigma) = \{\sigma' \in \text{Res}_0(\delta, \sigma) : \nexists \sigma'' \in \text{Res}_0(\delta, \sigma) (\text{New}(\delta, \sigma, \sigma'') \subset \text{New}(\delta, \sigma, \sigma'))\}.$$

Ostatecznie wynik akcji złożonej jest sumą wyników wszystkich jej dekompozycji:

$$\text{Res}(\mathbb{A}, \sigma) = \bigcup_{\delta \in \Delta(\mathbb{A}, \sigma)} \text{Res}(\delta, \sigma).$$

Pseudokod dla algorytmu przedstawiono w algorytmie 6.

Algorithm 6 Pseudokod dla algorytmu generującego funkcję Res dla akcji złożonej

```

1: procedure GENERUJRESZLOZONE( $\mathbb{A}, \sigma, D$ )
2:    $\Delta(\mathbb{A}, \sigma) \leftarrow \text{GenerujDekompozycje}(\mathbb{A}, \sigma, D)$ 
3:   if  $\Delta(\mathbb{A}, \sigma) = \emptyset$  then return  $\emptyset$ 
4:    $R_{\text{all}} \leftarrow \emptyset$ 
5:   for all  $\delta \in \Delta(\mathbb{A}, \sigma)$  do
6:     wyznacz  $E(\delta, \sigma)$  oraz  $R(\delta, \sigma)$ 
7:     zbuduj  $\text{Res}_0(\delta, \sigma)$ 
8:     for all  $\sigma' \in \text{Res}_0(\delta, \sigma)$  do
9:       oblicz  $\text{New}(\delta, \sigma, \sigma')$ 
10:    wyznacz  $\text{Res}(\delta, \sigma)$  przez minimalizację względem zawierania
11:     $R_{\text{all}} \leftarrow R_{\text{all}} \cup \text{Res}(\delta, \sigma)$ 
return  $R_{\text{all}}$ 

```

8.7 Ewaluacja kwerend

Ewaluację kwerend najwygodniej oprzeć na drzewie wykonania procesu. Korzeń takiego drzewa odpowiada wszystkim stanom początkowym z Σ_0 . Na poziomie i przechowujemy wszystkie stany osiągnięte po wykonaniu pierwszych i kroków procesu, a między kolejnymi poziomami zapisujemy relację przejścia wynikającą z funkcji Res.

Taka reprezentacja jest szczególnie istotna w przypadku kwerend typu **necessary**. Sam zbiór stanów osiągalnych po kolejnych krokach nie wystarcza, ponieważ może się zdarzyć, że część ścieżek zakończy się wcześniej brakiem następnika, mimo że inne ścieżki nadal prowadzą do stanu końcowego. Dlatego przy ewaluacji kwerend koniecznych należy badać całe drzewo wykonania, a nie tylko końcowy zbiór stanów.

Ponieważ w projektowanym języku kwerend rozważamy procesy rozpoczynane ze zbioru stanów początkowych Σ_0 , przyjmujemy następującą interpretację czterech podstawowych typów kwerend:

1. **possibly executable** $\mathbb{P}$,
2. **necessary executable** $\mathbb{P}$,
3. **possibly γ after** $\mathbb{P}$,
4. **necessary γ after** $\mathbb{P}$.

Kwerenda **possibly executable** jest prawdziwa wtedy i tylko wtedy, gdy istnieje co najmniej jedna pełna ścieżka wykonania procesu od pewnego stanu początkowego do poziomu n .

Kwerenda **necessary executable** jest prawdziwa wtedy i tylko wtedy, gdy każda ścieżka rozpoczęta w dowolnym stanie początkowym może zostać doprowadzona do końca procesu. Algorytmicznie oznacza to, że w drzewie wykonania nie istnieje węzeł na poziomie mniejszym niż n , który nie miałby żadnego następnika.

Kwerenda **possibly γ after** $\mathbb{P}$ jest prawdziwa wtedy i tylko wtedy, gdy istnieje przynajmniej jedna pełna ścieżka wykonania, której stan końcowy spełnia formułę γ .

Kwerenda **necessary γ after** $\mathbb{P}$ jest prawdziwa wtedy i tylko wtedy, gdy proces jest koniecznie wykonalny oraz każdy stan końcowy każdej pełnej ścieżki spełnia formułę γ . Przyjmujemy więc, że niewykonalność procesu na choć jednej ścieżce powoduje fałszywość tej kwerendy.

Pseudokod dla algorytmu przedstawiono w algorytmie 7.

Algorithm 7 Pseudokod dla algorytmu wykonującego kwerendę

```
1: procedure KWERENDA( $Q, D$ )
2:   zbuduj drzewo wykonania procesu od wszystkich stanów  $\Sigma_0$ 
3:   if pytanie ma postać possibly executable  $\mathbb{P}$  then return true wtedy i tylko wtedy,
     gdy istnieje pełna ścieżka długości  $n$ 
4:   if pytanie ma postać necessary executable  $\mathbb{P}$  then return true wtedy i tylko wtedy,
     gdy każda ścieżka dochodzi do poziomu  $n$ 
5:   if pytanie ma postać possibly  $\gamma$  after  $\mathbb{P}$  then return true wtedy i tylko wtedy, gdy
     istnieje pełna ścieżka kończąca się w stanie spełniającym  $\gamma$ 
6:   if pytanie ma postać necessary  $\gamma$  after  $\mathbb{P}$  then return true wtedy i tylko wtedy, gdy
     każda pełna ścieżka kończy się w stanie spełniającym  $\gamma$  oraz nie istnieje ścieżka zablokowana
     przed końcem procesu
```

8.8 Uwagi o złożoności

Niech:

- $m = |\mathcal{F}|$ oznacza liczbę fluentów,
- $N = |\Sigma| \leq 2^m$ liczbę stanów dopuszczalnych,
- $k = |\mathbb{A}|$ liczbę akcji prostych w pojedynczym kroku akcji złożonej,
- n długość procesu.

Wtedy:

- generacja Σ jest wykładnicza względem m ,
- obliczenie $\text{Res}(A, \sigma)$ dla akcji prostej jest wielomianowe względem N ,
- generacja dekompozycji akcji złożonej jest wykładnicza względem k ,
- ewaluacja całego procesu może być wykładnicza względem długości procesu, ponieważ liczba ścieżek niedeterministycznych może szybko rosnąć.

Z praktycznego punktu widzenia nie stanowi to jednak istotnego ograniczenia dla projektu. Rozważane przez nas przykłady zawierają niewielką liczbę fluentów i akcji, dlatego pełna eksploracja przestrzeni stanów, dekompozycji oraz ścieżek wykonania pozostaje w pełni akceptowalna. Jednocześnie taka implementacja jest przejrzysta, łatwa do uzasadnienia formalnie i bezpośrednio odpowiada definicjom semantycznym wprowadzonym wcześniej.

9 Decyzje projektowe i uzasadnienia

Poniżej zbieramy najważniejsze decyzje projektowe podjęte przy opracowywaniu klasy DS4 — zarówno miejsca, w których świadomie odchodzimy od treści zadania lub standardowej postaci języków AR/AC , jak i rozstrzygnięcia interpretacyjne przypadków brzegowych. Szczegółowe uzasadnienia znajdują się w odpowiednich sekcjach formalnych; tutaj prezentujemy je w skróconej formie.

9.1 Σ_0 zamiast σ_0

Klasyczny język AR posługuje się pojedynczym wyróżnionym stanem początkowym σ_0 . Warunek Z7 (dopuszczalność opisów częściowych) wymusza przejście do *zbioru* stanów początkowych Σ_0 : zdania **initially** mogą nie ustalać wartości wszystkich fluentów, więc z dziedziną może być zgodnych wiele stanów jednocześnie. Operacyjnie Σ_0 wyznaczamy z samych zdań **initially** (formuła w 4.2, pseudokod §8.1); pozostałe zdania specyfikacyjne — **after** oraz **observable after** — traktujemy jako warunki poprawności modelu (M2 w §6.4), a nie jako dodatkowy filtr stanów początkowych. Przeniesienie tej konstrukcji na język kwerend oznacza, że kwantyfikator „**necessary**” rozciąga się na wszystkie $\sigma_0 \in \Sigma_0$ (patrz §7.3); scenariusz YSP (§10.1), w którym nieustalona wartość *loaded* daje $|\Sigma_0| = 2$, ilustruje tę konstrukcję w praktyce.

9.2 Mechanizm AC zamiast rozłączności z treści projektu

Treść zadania definiuje wykonalność akcji złożonej przez warunek rozłączności zbiorów zmiennych, na które wpływają jej składowe. Przyjęliśmy ogólniejszy mechanizm zaczerpnięty z języka AC — detekcję konfliktów i generację dekompozycji (rozdział 5) — z trzech powodów: (a) rozłączność nie rozróżnia akcji pewnych (**causes**) i niedeterministycznych (**releases**), przez co nie obsługuje poprawnie Przypadku 2 konfliktu; (b) nie uwzględnia ramifikacji wymuszanych zdaniami **always**, które mogą rozszerzyć faktyczny zbiór zmienianych fluentów; (c) sam warunek rozłączności jest szczególnym przypadkiem mechanizmu AC (formalny argument w §5.6). Wybór ten został uzgodniony z prowadzącą przedmiot.

9.3 $\text{Res}(\mathbb{A}, \sigma) = \emptyset$ przy braku wykonalnych składowych

Jeżeli w stanie σ żadna akcja prosta $A_i \in \mathbb{A}$ nie jest indywidualnie wykonalna, to $V_{\text{exec}} = \emptyset$, a zbiór dekompozycji jest pusty (§8.5). Alternatywą byłoby zinterpretowanie akcji złożonej o pustej dekompozycji jako akcji identycznościowej ($\{\sigma\}$), co czyniłoby $\mathbb{A}$ „wykonalną” nawet wtedy, gdy żadna jej składowa nie jest wykonalna. Przyjmujemy odwrotną konwencję: $\Delta(\mathbb{A}, \sigma) = \emptyset \Rightarrow \text{Res}(\mathbb{A}, \sigma) = \emptyset$. Decyzja ta jest spójna z traktowaniem akcji złożonej jako zbioru działań wymagających jednoczesnego wykonania i stanowi wewnętrzne dopełnienie reguły priorytetu **impossible** na poziomie akcji prostej (patrz §9.4).

9.4 Semantyka przypadków brzegowych

Dwa przypadki brzegowe w semantyce wymagają jawnego rozstrzygnięcia, ponieważ formalna definicja pozostawia je niejednoznaczny.

Priorytet impossible nad efektem pustym. Jeżeli w stanie σ aktywne jest zdanie **impossible** A **if** π , a jednocześnie żadna reguła **causes** dla A nie ma spełnionego warunku, to wprost z definicji Res_0 otrzymalibyśmy $\text{Res}_0(A, \sigma) = \Sigma$, czyli akcja zachowywałaby się jak „nic nie robiąca”. Przyjmujemy, że **impossible** ma priorytet semantyczny: $\text{Res}(A, \sigma) = \emptyset$ niezależnie od reguł efektowych. Odpowiada to klasycznej interpretacji **impossible** A **if** π jako skrótu dla A **causes** $\perp$ **if** π (patrz §4.6, §8.2).

Kwerendy wartościowe a niewykonalność procesu. Jeżeli proces $\mathbb{P}$ nie jest wykonalny z pewnego $\sigma_0 \in \Sigma_0$, to funkcja Ψ nie jest dla niego określona. Przyjmujemy, że **necessary** γ **after** $\mathbb{P}$ jest wtedy *falszywa*, ponieważ nie można stwierdzić, iż γ zachodzi po wykonaniu procesu, który się nie kończy; symetrycznie **possibly** γ **after** $\mathbb{P}$ wymaga istnienia co najmniej jednej *pełnej* ścieżki kończącej się w stanie spełniającym γ . W szczególności **necessary** γ **after** $\mathbb{P}$ implikuje **necessary executable** $\mathbb{P}$. Odpowiadająca procedura obliczeniowa znajduje się w §8.7.

10 Przykłady

10.1 YSP z rosyjską ruletką

Scenariusz oparty na rosyjskim indyku, ilustrujący niedeterminizm (**releases**), niewykonalność (**impossible**) i częściowy opis stanu początkowego.

Sygnatura:

$$\mathcal{F} = \{loaded, alive\},$$

$$\mathcal{A} = \{Load, Shoot, Spin\}.$$

Dziedzina D :

initially <i>alive</i>	(stan broni nieznan — opis częściowy)
<i>Load</i> causes <i>loaded</i>	
impossible <i>Load</i> if <i>loaded</i>	(nie można ładować naładowanej)
<i>Shoot</i> causes $\neg loaded$	
<i>Shoot</i> causes $\neg alive$ if <i>loaded</i>	
<i>Spin</i> releases <i>loaded</i>	(kręcenie bębenkiem — niedeterminizm)
$\neg loaded$ after <i>Shoot</i>	(specyfikacja: po strzale broń rozładowana)

Stany dopuszczalne (brak **always**, wszystkie 4 stany dopuszczalne):

$$\sigma_0 = \{loaded, alive\}, \quad \sigma_1 = \{loaded, \neg alive\}, \quad \sigma_2 = \{\neg loaded, alive\}, \quad \sigma_3 = \{\neg loaded, \neg alive\}$$

Stany początkowe. Zdanie **initially** *alive* wybiera z Σ dwa stany: σ_0 oraz σ_2 (*loaded* nieokreślone — opis częściowy, Z7). Zdanie wartościowe $\neg loaded$ **after** *Shoot* weryfikujemy dla obu kandydatów: $\text{Res}(Shoot, \sigma_0) = \{\sigma_3\} \models \neg loaded$ (akcja rozładowuje i zabija) oraz $\text{Res}(Shoot, \sigma_2) = \{\sigma_2\} \models \neg loaded$ (efekt pusty — broń już rozładowana). Oba stany przechodzą warunek, więc $\Sigma_0 = \{\sigma_0, \sigma_2\}$ pozostaje bez zmian.

Proces $P = (\{Load\}, \{Spin\}, \{Shoot\})$.

Z σ_0 : *Load* jest niewykonalne (bo $\sigma_0 \models loaded$, a **impossible** *Load* **if** *loaded*). Proces nie może startować z σ_0 .

Z σ_2 :

1. *Load* w σ_2 : efekt *loaded* $\Rightarrow \sigma_0 = \{loaded, alive\}$.
2. *Spin* w σ_0 : fluent *loaded* jest zwolniony (**releases**). Oba stany σ_0 i σ_2 zmieniają się minimalnie, więc $\text{Res}(Spin, \sigma_0) = \{\sigma_0, \sigma_2\}$ (niedeterminizm).
3. *Shoot* po każdej gałęzi:
 - ze σ_0 (broń naładowana): efekty $\neg loaded \wedge \neg alive \Rightarrow \sigma_3$.
 - ze σ_2 (broń pusta): efekt $\neg loaded$ już spełniony, warunek **if** *loaded* fałszywy $\Rightarrow$ efekt pusty, σ_2 bez zmian.

Stany końcowe: $\{\sigma_2, \sigma_3\}$.

Kwerendy:

- **necessary executable** P — z $\sigma_0 \in \Sigma_0$ pierwszy krok jest niewykonalny. **NIE**.
- **possibly executable** P — z σ_2 proces przechodzi wszystkie kroki. **TAK**.
- **necessary** $\neg alive$ **after** P — gałąź przez σ_2 (po *Spin*) kończy się w σ_2 z *alive*. **NIE**.
- **possibly** $\neg alive$ **after** P — gałąź przez σ_0 (po *Spin*) kończy się w σ_3 . **TAK**.

10.2 Producent i konsumenci

Scenariusz oparty na przykładzie 5.8. W odróżnieniu od oryginału nie blokujemy sytuacji, w której obaj konsumenci pobierają produkt jednocześnie — w $\mathbb{DS4}$ nie można wyrazić **impossible** dla akcji złożonej. Daje to dodatkową dekompozycję i dodatkowy niedeterminizm.

Producent napełnia bufor, dwaj konsumenci A i B pobierają z niego produkt i zużywają go.

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{empty, hasA, hasB\}, \\ \mathcal{A} &= \{Put, GetA, GetB, ConsumeA, ConsumeB\}.\end{aligned}$$

Dziedzina D :

initially $\neg empty \wedge hasA \wedge hasB$	(bufor pełny, obaj konsumenci z produktem)
Put causes $\neg empty$	(producent napełnia bufor)
GetA causes $empty \wedge hasA$ if $\neg empty$	(A pobiera, o ile jest co pobrać)
GetB causes $empty \wedge hasB$ if $\neg empty$	(B pobiera, o ile jest co pobrać)
ConsumeA causes $\neg hasA$ if $hasA$	(A zużywa swój produkt)
ConsumeB causes $\neg hasB$ if $hasB$	(B zużywa swój produkt)

Brak **always** — wszystkie $2^3 = 8$ stanów jest dopuszczalnych. Stan początkowy: $\Sigma_0 = \{\sigma\}$ gdzie $\sigma = \{\neg empty, hasA, hasB\}$ (bufor pełny, obaj konsumenci mają produkt).

Akcja złożona $\mathbb{A} = \{Put, GetA, GetB, ConsumeA, ConsumeB\}$ w σ .

Konflikty w σ (oba warunki wstępne spełnione):

- *Put* vs. *GetA*: *Put* daje $\neg empty$, *GetA* daje $empty \wedge hasA$ — literały $\neg empty$ i $empty$ są komplementarne. **Konflikt** (Przypadek 1).
- *Put* vs. *GetB*: analogicznie. **Konflikt**.
- *GetA* vs. *ConsumeA*: *GetA* daje $empty \wedge hasA$, *ConsumeA* daje $\neg hasA$ — $hasA$ i $\neg hasA$ komplementarne. **Konflikt**.
- *GetB* vs. *ConsumeB*: analogicznie. **Konflikt**.
- *GetA* vs. *GetB*: oba dają $empty$ (ten sam znak) i różne fluenty $hasA, hasB$ — **brak konfliktu**. W oryginalnym przykładzie 5.8 *GetA* i *GetB* są wykluczone zdaniem **impossible** $\{GetA, GetB\}$. W $\mathbb{DS4}$ **impossible** dotyczy tylko akcji prostych, więc tej pary nie możemy tak zablokować.
- *ConsumeA* vs. *ConsumeB*, *Put* vs. *ConsumeA/ConsumeB*, *GetA* vs. *ConsumeB*, *GetB* vs. *ConsumeA*: brak konfliktu.

Dekompozycje — maksymalne podzbiory bez konfliktu:

- $\Delta_1 = \{Put, ConsumeA, ConsumeB\}$ — nie da się dołożyć *GetA* ani *GetB* (oba kolidują z *Put*).
- $\Delta_2 = \{GetA, GetB\}$ — nie da się dołożyć *Put*, *ConsumeA* ani *ConsumeB* (konflikty). *Ta dekompozycja nie istnieje w oryginalnym przykładzie 5.8.*
- $\Delta_3 = \{GetA, ConsumeB\}$ — nie da się dołożyć *Put*, *GetB* ani *ConsumeA* (konflikty).
- $\Delta_4 = \{GetB, ConsumeA\}$ — analogicznie.

Wyniki z $\sigma = \{\neg empty, hasA, hasB\}$:

- $\Delta_1: \neg \text{empty} \wedge \neg \text{has}A \wedge \neg \text{has}B \Rightarrow \sigma_1$.
- $\Delta_2: \text{empty} \wedge \text{has}A \wedge \text{has}B \Rightarrow \sigma_2$.
- $\Delta_3: \text{empty} \wedge \text{has}A \wedge \neg \text{has}B \Rightarrow \sigma_3$.
- $\Delta_4: \text{empty} \wedge \neg \text{has}A \wedge \text{has}B \Rightarrow \sigma_4$.

$\text{Res}(\mathbb{A}, \sigma) = \{\sigma_1, \sigma_2, \sigma_3, \sigma_4\}$. Oryginalne AC daje tylko $\{\sigma_1, \sigma_3, \sigma_4\}$ — brak σ_2 , bo tam $\text{Get}A$ i $\text{Get}B$ są razem niemożliwe.

Proces $P = (\mathbb{A}, \{\text{Consume}A, \text{Consume}B\})$.

Krok 2 — $\{\text{Consume}A, \text{Consume}B\}$ w każdym stanie wynikowym (brak konfliktu, bo $\neg \text{has}A$ i $\neg \text{has}B$ dotyczą różnych fluentów):

- $\sigma_1 = \{\neg e, \neg A, \neg B\}$: oba warunki ($\text{has}A, \text{has}B$) niespełnione — efekt pusty, stan bez zmian.
- $\sigma_2 = \{e, A, B\}$: obaj konsumenci konsumują $\Rightarrow \sigma_0 = \{e, \neg A, \neg B\}$.
- $\sigma_3 = \{e, A, \neg B\}$: tylko A konsumuje $\Rightarrow \sigma_0$.
- $\sigma_4 = \{e, \neg A, B\}$: tylko B konsumuje $\Rightarrow \sigma_0$.

(Używamy skrótu $e = \text{empty}$, $A = \text{has}A$, $B = \text{has}B$.)

Po procesie P : $\text{Res} = \{\sigma_0, \sigma_1\}$.

Kwerendy:

- **necessary executable** P — oba kroki mają $\text{Res} \neq \emptyset$. **TAK**.
- **possibly empty after** P — σ_0 spełnia empty . **TAK**.
- **necessary empty after** P — σ_1 spełnia $\neg \text{empty}$. **NIE**.
- **necessary $\neg \text{has}A \wedge \neg \text{has}B$ after** P — zarówno σ_0 , jak i σ_1 spełniają $\neg \text{has}A \wedge \neg \text{has}B$. **TAK** — niezależnie od przebiegu, po procesie żaden konsument nie ma produktu.

10.3 Dwa przełączniki, żarówka i alarm

Nowy przykład rozszerzający dwuprzełącznikowy scenariusz. Dwa przełączniki sw_1, sw_2 sterują żarówką (światło świeci, gdy włączony jest przynajmniej jeden z nich), a gdy świeci światło, uruchamia się alarm (czujnik ruchu). Dodatkowo akcja On wymusza zapalenie światła (np. ręczne zwarcie obwodu), a akcja $Flicker$ modeluje zakłócenie zasilania — nie wymusza konkretnej wartości $light$, lecz zwalnia ten fluent spod inercji.

Scenariusz demonstruje: **always** (ograniczenia integralności i ramifikacje), **noninertial** (alarm zmienia się „za darmo” jako efekt uboczny), akcję złożoną bez konfliktu oraz konflikt Przypadek 2 (**causes** vs. **releases**).

Sygnatura:

$$\begin{aligned}\mathcal{F} &= \{sw_1, sw_2, light, alarm\}, \\ \mathcal{A} &= \{Toggle_1, Toggle_2, On, Flicker\}.\end{aligned}$$

Dziedzina D :

always $light \leftrightarrow (sw_1 \vee sw_2)$	(ramifikacja: światło świeci, gdy włączony choć jeden przełącznik)
always $light \rightarrow alarm$	(ramifikacja: świecąca żarówka aktywuje alarm)
noninertial $alarm$	(alarm nie podlega inercji — reaguje na światło)
$Toggle_1$ causes sw_1 if $\neg sw_1$	(przełącznik pierwszy: warunkowe causes)
$Toggle_1$ causes $\neg sw_1$ if sw_1	
$Toggle_2$ causes sw_2 if $\neg sw_2$	(przełącznik drugi: warunkowe causes)
$Toggle_2$ causes $\neg sw_2$ if sw_2	
On causes $light$	(wymuszenie zapalenia światła)
$Flicker$ releases $light$	(zakłócenie — światło może się zmienić)
initially $\neg sw_1 \wedge \neg sw_2$	(stan $alarm$ nieokreślony — opis częściowy)

Stany dopuszczalne Σ . Z $2^{|\mathcal{F}|} = 16$ wartościowań odrzucamy te, które łamią zdania **always**. Po pierwsze, musi zachodzić $light \leftrightarrow (sw_1 \vee sw_2)$, co wyklucza 8 stanów i wyznacza wartość $light$ na podstawie przełączników. Po drugie, gdy $light$ jest prawdziwe, musi być prawdziwy również $alarm$, co eliminuje jeszcze 3 stany. Pozostaje 5 stanów dopuszczalnych:

$$\begin{aligned}
\sigma_0 &= \{\neg sw_1, \neg sw_2, \neg light, \neg alarm\}, \\
\sigma_1 &= \{\neg sw_1, \neg sw_2, \neg light, alarm\}, \\
\sigma_2 &= \{\neg sw_1, sw_2, light, alarm\}, \\
\sigma_3 &= \{sw_1, \neg sw_2, light, alarm\}, \\
\sigma_4 &= \{sw_1, sw_2, light, alarm\}.
\end{aligned}$$

Zdanie **initially** wymusza $\neg sw_1, \neg sw_2$, natomiast nie ustala wartości fluentu $alarm$. Z dopuszczalnych stanów warunek ten spełniają zatem dwa: σ_0 oraz σ_1 . Ostatecznie $\Sigma_0 = \{\sigma_0, \sigma_1\}$ — jest to przypadek opisu częściowego (Z7).

Proces $P = (\{Toggle_1, Toggle_2\}, \{On, Flicker\})$.

Krok 1: $\{Toggle_1, Toggle_2\}$ w σ_0 .

W stanie σ_0 aktywne są reguły $Toggle_i$ **causes** sw_i **if** $\neg sw_i$ (obu przełączników). Obie akcje są indywidualnie wykonalne.

Konflikty: efekty aktywne to sw_1 (DNF: $\{\{sw_1\}\}$) i sw_2 (DNF: $\{\{sw_2\}\}$). Literały dotyczą różnych fluentów, więc nie ma pary komplementarnej — brak konfliktu Przypadek 1. Żadna z akcji nie jest **releases**, więc brak Przypadek 2.

Dekompozycja: jedna, $\delta = \{Toggle_1, Toggle_2\}$.

Wynik: $E(\delta, \sigma_0) = \{sw_1, sw_2\}$, $R(\delta, \sigma_0) = \emptyset$.

$$Res_0(\delta, \sigma_0) = \{\sigma \in \Sigma : \sigma \models sw_1 \wedge sw_2\} = \{\sigma_4\}.$$

Jedyny kandydat, więc $Res(\{Toggle_1, Toggle_2\}, \sigma_0) = \{\sigma_4\}$ z $New(\sigma_0, \sigma_4) = \{sw_1, sw_2, light\}$. Zmiana $light$ jest wymuszona przez **always** (ramifikacja), zmiana $alarm$ jest „za darmo” — fluent nieinercyjny nie trafia do New .

Analogicznie dla drugiego stanu początkowego: w $\sigma_1 = \{\neg sw_1, \neg sw_2, \neg light, alarm\}$ aktywne reguły i zbiór $E(\delta, \sigma_1) = \{sw_1, sw_2\}$ są identyczne, więc $Res_0(\delta, \sigma_1) = \{\sigma_4\}$ i $Res(\{Toggle_1, Toggle_2\}, \sigma_1) = \{\sigma_4\}$. Obie gałęzie procesu zbiegają się zatem w σ_4 już po pierwszym kroku — nieokreśloność stanu początkowego alarmu została „wchłonięta” przez ramifikację $light \rightarrow alarm$ w nowym stanie.

Krok 2: $\{On, Flicker\}$ w σ_4 .

Indywidualna wykonalność:

- $\text{Res}(\{On\}, \sigma_4)$: $E = \{light\}$, $R = \emptyset$. $\text{Res}_0 = \{\sigma_2, \sigma_3, \sigma_4\}$ (wszystkie stany z *light*). Obliczenia New:

$$\text{New}(\sigma_4, \sigma_2) = \{\neg sw_1\}, \quad \text{New}(\sigma_4, \sigma_3) = \{\neg sw_2\}, \quad \text{New}(\sigma_4, \sigma_4) = \emptyset.$$

Minimum to $\emptyset$, zatem $\text{Res}(\{On\}, \sigma_4) = \{\sigma_4\}$ (akcja bez efektu — światło już świeci).

- $\text{Res}(\{Flicker\}, \sigma_4)$: $E = \emptyset$, $R = \{light\}$. $\text{Res}_0 = \Sigma = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4\}$. Fluent *light* jest zwolniony — jego literal (zgodny z σ') wchodzi do New:

$$\begin{aligned} \text{New}(\sigma_4, \sigma_0) &= \{\neg sw_1, \neg sw_2, \neg light\}, \\ \text{New}(\sigma_4, \sigma_1) &= \{\neg sw_1, \neg sw_2, \neg light\}, \\ \text{New}(\sigma_4, \sigma_2) &= \{\neg sw_1, light\}, \\ \text{New}(\sigma_4, \sigma_3) &= \{\neg sw_2, light\}, \\ \text{New}(\sigma_4, \sigma_4) &= \{light\}. \end{aligned}$$

Minimum względem inkluzji to $\{light\}$ (dla σ_4) oraz $\{\neg sw_1, \neg sw_2, \neg light\}$ (dla σ_0 i σ_1 , nieporównywalne z $\{light\}$). Zatem $\text{Res}(\{Flicker\}, \sigma_4) = \{\sigma_0, \sigma_1, \sigma_4\}$.

Konflikty: $E(On, \sigma_4) = \{light\}$, $\text{DNF}(light) = \{\{light\}\}$; *Flicker* zwalnia fluent *light*, który występuje w tym składniku — to **konflikt Przypadek 2 (causes vs. releases)**.

Dekompozycje: graf konfliktu na $V_{\text{exec}} = \{On, Flicker\}$ ma jedną krawędź $\{On, Flicker\}$. Maksymalne zbiory bezkonfliktowe to dwa singletony: $\delta_1 = \{On\}$, $\delta_2 = \{Flicker\}$.

Wynik:

$$\text{Res}(\{On, Flicker\}, \sigma_4) = \text{Res}(\delta_1, \sigma_4) \cup \text{Res}(\delta_2, \sigma_4) = \{\sigma_4\} \cup \{\sigma_0, \sigma_1, \sigma_4\} = \{\sigma_0, \sigma_1, \sigma_4\}.$$

Stany osiągalne po procesie P : $\{\sigma_0, \sigma_1, \sigma_4\}$.

Kwerendy:

- **necessary executable P** — Z obu stanów początkowych σ_0, σ_1 krok 1 prowadzi do σ_4 , a krok 2 ma w σ_4 niepuste Res, więc każda ścieżka dochodzi do poziomu 2. **TAK**.
- **possibly executable P** — Natychmiast z powyższego. **TAK**.
- **necessary alarm after P** — σ_0 spełnia $\neg alarm$, więc istnieje ścieżka kończąca się bez alarmu. **NIE**.
- **possibly alarm after P** — σ_4 i σ_1 spełniają *alarm*. **TAK**.

Komentarz do ramifikacji i nieinercji. W kroku 1 jedyna zmiana wymuszona bezpośrednio przez **causes** to sw_1, sw_2 ; zmiana *light* wynika z **always** i zostaje wliczona do New, natomiast *alarm* także się zmienia „w tle”, ale dzięki **noninertial** nie obciąża zbioru New. W kroku 2 *Flicker* pokazuje, że **releases** może dawać więcej niż jeden minimalny stan wynikowy nawet pojedynczo, a w połączeniu z *On* generuje konflikt Przypadek 2, co rozбивa akcję złożoną na dwie singletonowe dekompozycje.

10.4 Dwóch robotników wynoszących stół

Autorski scenariusz integrujący, obejmujący wszystkie mechanizmy dziedziny $\mathbb{DS4}$: opis częściowy (Z7), ograniczenia integralności, fluenty nieinercyjne, ramifikacje, warunkowe **causes** i **releases**, **impossible** aktywowaną dynamicznie, obie kategorie konfliktu akcji złożonej (Przypadek 1 i Przypadek 2) oraz zdanie obserwacyjne przyczynające zbiór Σ_0 . Na jednej dziedzinie porównamy dwa procesy P_1 i P_2 , tak by w sumie trafić wszystkie interesujące ścieżki.

Dwóch robotników, A i B , ma wynieść stół z pokoju. Każdy może samodzielnie go podnosić lub puszczać ($LiftA/B$, $DropA/B$), otwierać lub zamykać drzwi ($OpenA/B$, $CloseA/B$) oraz wspólnie wynieść stół na zewnątrz ($CarryOut$). Stół uznajemy za uniesiony tylko wtedy, gdy trzymają go jednocześnie obaj robotnicy. Drzwi można obsługiwać wyłącznie wolną ręką.

Sygnatura:

$$\mathcal{F} = \{holdA, holdB, tableUp, doorOpen, broken, outside\},$$

$$\mathcal{A} = \{LiftA, LiftB, DropA, DropB, OpenA, OpenB, CloseA, CloseB, CarryOut\}.$$

Dla zwięzłości poniżej używamy skrótów $a = holdA$, $b = holdB$, $u = tableUp$, $d = doorOpen$, $r = broken$, $o = outside$ i zapisujemy stan jako szóstkę (a, b, u, d, r, o) .

Dziedzina D :

Ramifikacje i fluenty nieinercyjne:

$$\text{always } u \leftrightarrow (a \wedge b) \quad (\text{stół uniesiony} \Leftrightarrow \text{trzymają go obaj})$$

$$\text{noninertial } u \quad (\text{stół zależy od robotników})$$

Podnoszenie i opuszczanie stołu:

$$LiftA \text{ causes } a, \quad LiftB \text{ causes } b \quad (\text{pojedynczy robotnik chwyta stół})$$

$$DropA \text{ causes } \neg a, \quad DropB \text{ causes } \neg b \quad (\text{pojedynczy robotnik puszcza stół})$$

Obsługa drzwi (tylko wolną ręką):

$$OpenA \text{ causes } d, \quad OpenB \text{ causes } d \quad (\text{dowolny robotnik otwiera drzwi})$$

$$CloseA \text{ causes } \neg d, \quad CloseB \text{ causes } \neg d \quad (\text{dowolny robotnik zamyka drzwi})$$

$$\text{impossible } OpenA \text{ if } a, \quad \text{impossible } OpenB \text{ if } b \quad (\text{potrzeba wolnej ręki})$$

$$\text{impossible } CloseA \text{ if } a, \quad \text{impossible } CloseB \text{ if } b \quad (\text{analogicznie dla zamykania})$$

Wynoszenie stołu:

$$CarryOut \text{ causes } o \text{ if } d \quad (\text{wyjście tylko przez otwarte drzwi})$$

$$\text{impossible } CarryOut \text{ if } \neg u \quad (\text{nie można wynieść leżącego stołu})$$

$$CarryOut \text{ releases } a \text{ if } \neg d$$

$$CarryOut \text{ releases } b \text{ if } \neg d$$

$$CarryOut \text{ releases } r \text{ if } \neg d \quad (\text{uderzenie w drzwi — chaos})$$

Opis stanu początkowego i obserwacje:

$$\text{initially } \neg a \wedge \neg b \wedge \neg o \quad (\text{nie trzymają, stół w środku; } d, r \text{ nieustalone})$$

$$\text{observable } \neg r \text{ after } LiftA, LiftB, CarryOut \quad (\text{obserwacja — da się wynieść cało})$$

Stany dopuszczalne Σ . Spośród $2^6 = 64$ wartościowań ograniczenie **always** $u \leftrightarrow (a \wedge b)$ wyznacza u na podstawie a, b , co daje $|\Sigma| = 2^5 = 32$ stany (swobodne a, b, d, r, o).

Zbiór Σ_0 — krok 1: zdanie initially. Warunek $\neg a \wedge \neg b \wedge \neg o$ ustala trzy fluenty; u jest wyliczone ($u = F$ dla $a = b = F$). Swobodne pozostają d, r , co daje cztery kandydatury:

$$s_1 = (F, F, F, F, F, F), \quad s_2 = (F, F, F, F, T, F),$$

$$s_3 = (F, F, F, T, F, F), \quad s_4 = (F, F, F, T, T, F).$$

Zbiór Σ_0 — krok 2: przycięcie zdaniem obserwacyjnym. Zdanie **observable** $\neg r$ **after** $LiftA, LiftB, CarryOut$ wymaga, by *istniała* ścieżka wykonania sekwencji w której stan końcowy spełnia $\neg r$. W każdym kandydacie s_i po $LiftA, LiftB$ mamy $a = b = u = T$, wartości d, r niezmienione. W kroku $CarryOut$:

- gdy $d = F$ (stany s_1, s_2): $\neg d$ aktywuje **releases** dla a, b, r , natomiast **causes** o **if** d jest nieaktywne. Fluent r jest zwolniony, więc istnieje stan wynikowy z $r = F$ — warunek obserwacyjny spełniony.

- gdy $d = T$ (stany s_3, s_4): **releases** nieaktywne, r pozostaje bez zmian przez inercję. s_3 ma $r = F$ i warunek spełnia; s_4 ma $r = T$, a jedyny stan wynikowy też ma $r = T$ — warunek zawodzi.

Odrzucamy więc wyłącznie s_4 . Ostatecznie

$$\Sigma_0 = \{\sigma_0, \sigma_1, \sigma_2\}, \quad \sigma_0 = s_1, \quad \sigma_1 = s_2, \quad \sigma_2 = s_3.$$

Jest to przypadek opisu częściowego (Z7): trzy stany początkowe, różniące się wartością d (drzwi) i r (stan stołu) poza zakresem **initially**.

Poniżej rozważymy dwa procesy, które razem pokazują wszystkie mechanizmy semantyki. Każdy z nich używamy jako $\mathbb{P}$ w osobnym komplecie kwerend.

Proces $P_1 = (\{OpenA\}, \{LiftA, DropA\}, \{LiftB\}, \{CarryOut\})$. Cel: pokazać (i) akcję singletonową z warunkową **causes**, (ii) konflikt Przypadek 1 w kroku 2, (iii) dynamiczne **impossible** aktywujące się dopiero na końcu procesu.

Krok 1: $\{OpenA\}$. Singleton — brak konfliktu. W każdym $\sigma \in \Sigma_0$ mamy $a = F$, więc **impossible** $OpenA$ **if** a jest nieaktywne.

- $\sigma_0 \rightarrow \sigma_2$ (jedyna zmiana: $d: F \rightarrow T$).
- $\sigma_1 \rightarrow \sigma_3 := (F, F, F, T, T, F)$.
- $\sigma_2 \rightarrow \sigma_2$ (d już prawdziwe — efekt pusty, $New = \emptyset$).

Zbiór osiągalny: $\{\sigma_2, \sigma_3\}$.

Krok 2: $\{LiftA, DropA\}$. W obu stanach obie akcje są indywidualnie wykonalne. $E(LiftA) = \{a\}$, $DNF = \{\{a\}\}$; $E(DropA) = \{\neg a\}$, $DNF = \{\{\neg a\}\}$. Literały a i $\neg a$ są komplementarne — **konflikt Przypadek 1**. Dekompozycje: $\delta_1 = \{LiftA\}$, $\delta_2 = \{DropA\}$.

Wyniki (stan zapisujemy przed $\rightarrow$ po):

- z σ_2 : δ_1 daje $\sigma_4 := (T, F, F, T, F, F)$; δ_2 daje σ_2 (efekt $\neg a$ już spełniony). $Res = \{\sigma_2, \sigma_4\}$.
- z σ_3 : analogicznie $\sigma_5 := (T, F, F, T, T, F)$ oraz σ_3 . $Res = \{\sigma_3, \sigma_5\}$.

Krok 3: $\{LiftB\}$. Singleton — brak konfliktu, brak **impossible**. Akcja wymusza b ; ramifikacja **always** dopasowuje u do $a \wedge b$ (tutaj u nieinercyjny, więc nie trafia do New):

- $\sigma_2 \rightarrow \sigma_6 := (F, T, F, T, F, F)$ (u pozostaje F , bo $a = F$).
- $\sigma_3 \rightarrow \sigma_7 := (F, T, F, T, T, F)$.
- $\sigma_4 \rightarrow \sigma_8 := (T, T, T, T, F, F)$ (u wchodzi w T przez ramifikację).
- $\sigma_5 \rightarrow \sigma_9 := (T, T, T, T, T, F)$.

Krok 4: $\{CarryOut\}$. Singleton, ale **impossible** $CarryOut$ **if** $\neg u$ blokuje akcję w stanach z $u = F$:

- σ_6, σ_7 : $u = F$ — **niewykonalne**. Ślad Ψ na tej gałęzi pozostaje niezdefiniowany.
- σ_8 : $u = T$, $d = T$, więc **causes** o **if** d wymusza o , **releases** (**if** $\neg d$) nieaktywne. $Res = \{\sigma_{10}\}$, $\sigma_{10} := (T, T, T, T, F, T)$.
- σ_9 : analogicznie, $Res = \{\sigma_{11}\}$, $\sigma_{11} := (T, T, T, T, T, T)$.

Drzewo śladów P_1 :

$$\begin{aligned}
&\sigma_0 \rightarrow \sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_6 \rightarrow \textbf{niewykonalne} \\
&\quad \sigma_4 \rightarrow \sigma_8 \rightarrow \sigma_{10} \text{ (SUCCESS)} \\
&\sigma_1 \rightarrow \sigma_3 \rightarrow \sigma_3 \rightarrow \sigma_7 \rightarrow \textbf{niewykonalne} \\
&\quad \sigma_5 \rightarrow \sigma_9 \rightarrow \sigma_{11} \text{ (SUCCESS)} \\
&\sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_2 \rightarrow \sigma_6 \rightarrow \textbf{niewykonalne} \\
&\quad \sigma_4 \rightarrow \sigma_8 \rightarrow \sigma_{10} \text{ (SUCCESS)}
\end{aligned}$$

Kwerendy dla P_1 :

- **necessary executable** P_1 — z każdego $\sigma \in \Sigma_0$ istnieje ślad kończący się „niewykonalne” (gałąź przez $\delta_2 = \{DropA\}$). **NIE**.
- **possibly executable** P_1 — z każdego $\sigma \in \Sigma_0$ istnieje ślad pełny (gałąź przez $\delta_1 = \{LiftA\}$). **TAK**.
- **necessary o after** P_1 — na gałęziach niewykonalnych Ψ jest niezdefiniowane, więc kwerenda konieczności nie jest spełniona. **NIE**.
- **possibly o after** P_1 — stany σ_{10}, σ_{11} spełniają o . **TAK**.

Proces $P_2 = (\{LiftA, LiftB\}, \{LiftA, CarryOut\})$. Cel: pokazać (i) akcję złożoną bez konfliktu w kroku 1, (ii) konflikt Przypadek 2 (**causes** vs. aktywne **releases**) w kroku 2, (iii) warunkową **releases** rozbijającą świat na wiele stanów wynikowych.

Krok 1: $\{LiftA, LiftB\}$. $E(LiftA) = \{a\}$, $E(LiftB) = \{b\}$ — różne fluenty, żadna akcja nie jest **releases**, więc **brak konfliktu** (ani Przypadek 1, ani 2). Jedna dekompozycja $\delta = \{LiftA, LiftB\}$, $E(\delta) = \{a, b\}$, $R(\delta) = \emptyset$.

Z każdego stanu $\sigma \in \Sigma_0$ otrzymujemy jeden stan wynikowy (ramifikacja wymusza $u = T$, fluent nieinercyjny):

- $\sigma_0 \rightarrow \sigma_{12} := (T, T, T, F, F, F)$.
- $\sigma_1 \rightarrow \sigma_{13} := (T, T, T, F, T, F)$.
- $\sigma_2 \rightarrow \sigma_8$ (ten sam σ_8 co w P_1).

Krok 2: $\{LiftA, CarryOut\}$. Tu dzieje się najwięcej. Analiza zależy od tego, czy d jest prawdziwe w stanie, w którym wykonujemy akcję złożoną.

W stanach σ_{12}, σ_{13} ($d = F$). *CarryOut*: $u = T$, akcja wykonalna; $\neg d = T$, więc *aktywna* jest **releases** dla a, b, r , natomiast **causes o if** d jest *nieaktywne*. Zatem $E(CarryOut) = \emptyset$, $R(CarryOut) = \{a, b, r\}$. *LiftA*: $E = \{a\}$, DNF = $\{\{a\}\}$ — fluent a występuje w DNF zarówno w efekcie **causes LiftA**, jak i w aktywnej **releases CarryOut**. **Konflikt Przypadek 2 (causes vs. releases)**.

Dekompozycje: $\delta_1 = \{LiftA\}$, $\delta_2 = \{CarryOut\}$.

- δ_1 w σ_{12} : a już prawdziwe, $New = \emptyset$, $Res(\delta_1, \sigma_{12}) = \{\sigma_{12}\}$; analogicznie $Res(\delta_1, \sigma_{13}) = \{\sigma_{13}\}$.
- δ_2 w σ_{12} : $E = \emptyset$, $R = \{a, b, r\}$. Fluenty niezwolnione (d, o) muszą pozostać, czyli $d = o = F$; fluenty zwolnione (a, b, r) mogą się zmieniać swobodnie; u wyliczane przez ramifikację. Dla każdej z $2^3 = 8$ kombinacji (a, b, r) mamy $New = \emptyset$ (żaden z fluentów inercyjnych nie wymusza zmiany). Wszystkie 8 kandydatów jest minimalnych:

$$\begin{aligned}
\tau_1 &= (T, T, T, F, T, F) = \sigma_{13}, & \tau_2 &= (T, T, T, F, F, F) = \sigma_{12}, \\
\tau_3 &= (T, F, F, F, T, F), & \tau_4 &= (T, F, F, F, F, F), \\
\tau_5 &= (F, T, F, F, T, F), & \tau_6 &= (F, T, F, F, F, F), \\
\tau_7 &= (F, F, F, F, T, F) = \sigma_1, & \tau_8 &= (F, F, F, F, F, F) = \sigma_0.
\end{aligned}$$

$\text{Res}(\delta_2, \sigma_{12}) = \{\tau_1, \dots, \tau_8\}$; tak samo $\text{Res}(\delta_2, \sigma_{13}) = \{\tau_1, \dots, \tau_8\}$ (zbiór zwolnionych fluentów pokrywa wszystkie fluenty, którymi oba stany się różnią).

$\text{Res}(\{LiftA, CarryOut\}, \sigma_{12}) = \text{Res}(\delta_1, \sigma_{12}) \cup \text{Res}(\delta_2, \sigma_{12}) = \{\tau_1, \dots, \tau_8\}$ (8 stanów, wszystkie z $o = F$). Analogicznie dla σ_{13} .

W stanie σ_8 ($d = T$). Teraz $\neg d = F$, więc wszystkie trzy **releases** są *nieaktywne*; **causes** *o* if d *aktywne*. $E(CarryOut) = \{o\}$, $R(CarryOut) = \emptyset$. Z $LiftA$ brak wspólnych literalów komplementarnych ani zwolnionych fluentów z DNF — **brak konfliktu**. Jedna dekompozycja $\delta = \{LiftA, CarryOut\}$, $E(\delta) = \{a, o\}$. Stan σ_8 już spełnia a ; jedyny minimalny stan wynikowy to σ_{10} (z P_1). Zatem $\text{Res}(\{LiftA, CarryOut\}, \sigma_8) = \{\sigma_{10}\}$.

Drzewo śladów P_2 :

$$\begin{aligned} \sigma_0 &\rightarrow \sigma_{12} \rightarrow \{\tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8\} \quad (\text{wszystkie mają } o = F) \\ \sigma_1 &\rightarrow \sigma_{13} \rightarrow \{\tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8\} \\ \sigma_2 &\rightarrow \sigma_8 \rightarrow \{\sigma_{10}\} \quad (\text{ma } o = T) \end{aligned}$$

Każdy ślad Ψ jest zdefiniowany — proces P_2 jest wszędzie wykonalny.

Kwerendy dla P_2 :

- **necessary executable P_2** — w żadnym węźle drzewa wykonania nie zachodzi blokada. **TAK**.
- **possibly executable P_2** — natychmiast z powyższego. **TAK**.
- **necessary o after P_2** — gałąź startująca z σ_0 kończy się w zbiorze $\{\tau_1, \dots, \tau_8\}$, w którym wszędzie $o = F$. Decyduje o tym wybór dekompozycji $\delta_2 = \{CarryOut\}$ w kroku 2 — bo $d = F$, **causes** dla o nie odpala, a **releases** nie wymusza o . **NIE**.
- **possibly o after P_2** — gałąź z σ_2 kończy się w σ_{10} z $o = T$ (drzwi początkowo otwarte, **causes o if d aktywne**). **TAK**.

Komplementarność procesów: P_1 prowadzi przez konflikt Przypadek 1 i aktywuje dynamiczną niewykonalność $CarryOut$, co daje negatywną odpowiedź na **necessary executable**; P_2 ujawnia konflikt Przypadek 2 z niedeterministyczną **releases**, która przy $d = F$ rozбивa świat na osiem stanów wynikowych i blokuje pozytywną odpowiedź na **necessary o after P_2** . Zdanie **observable** przyczynające Σ_0 demonstruje się już na poziomie dziedziny, wspólnie dla obu procesów.

11 Podsumowanie i kierunek dalszych prac

W niniejszym opracowaniu zaprojektowano kompletny formalny aparat do opisu oraz analizy systemów dynamicznych klasy DS4. W szczególności zdefiniowano:

- **język akcji**, umożliwiający formalny opis dziedziny w oparciu o akcje proste, ich efekty, warunki wykonalności oraz ograniczenia integralności,
- **semantykę operacyjną**, obejmującą zarówno pojedyncze akcje, jak i akcje złożone, wraz z mechanizmem konfliktów, dekompozycji oraz minimalizacji zmian,
- **model dziedziny**, który jednoznacznie określa zbiór stanów dopuszczalnych, stanów początkowych oraz funkcję przejścia,
- **język kwerend**, pozwalający formułować pytania dotyczące wykonalności procesów oraz osiągalności określonych stanów końcowych.

Kluczowym elementem projektu jest integracja mechanizmów znanych z języków *AR* oraz *AC* w ramach jednej spójnej semantyki. W szczególności wykorzystano: prawo inercji i rami-fikacje do modelowania zmian stanu, niedeterminizm wynikający z konstrukcji **releases**, oraz mechanizm dekompozycji akcji złożonych do obsługi współbieżności. Zaproponowane podejście uogólnia prosty model rozłączności zmiennych, umożliwiając bardziej elastyczne i realistyczne modelowanie konfliktów między akcjami.

Istotnym rozszerzeniem względem klasycznego modelu *AR* jest dopuszczenie wielu stanów początkowych (Σ_0), co wynika bezpośrednio z założenia częściowej informacji (Z7). Wprowadza to dodatkowy poziom niedeterminizmu, który musi być uwzględniany przy interpretacji kwerend.

Kierunki dalszych prac.

Kolejnym etapem projektu będzie implementacja zaproponowanego modelu oraz eksperymentalna weryfikacja jego działania. Planowane prace obejmują:

- **implementację parsera języka dziedziny i kwerend** – opracowanie prostego DSL umożliwiającego wprowadzanie specyfikacji w formie tekstowej,
- **generację przestrzeni stanów** (Σ oraz Σ_0) na podstawie ograniczeń i zdań początkowych,
- **implementację funkcji Res** dla akcji prostych i złożonych, w tym:
 - obliczanie Res_0 ,
 - wyznaczanie zbioru zmian New ,
 - minimalizację względem zasady inercji,
 - wykrywanie konfliktów i generację dekompozycji,
- **implementację ewaluacji procesów** (funkcja Ψ) wraz z obsługą niedeterminizmu,
- **obsługę kwerend** typu **necessary** oraz **possibly**, zarówno dla wykonalności, jak i osiągalności celów,
- **opracowanie zestawu testów i przykładów**, pozwalających zweryfikować poprawność implementacji na scenariuszach omawianych w pracy,
- opcjonalnie: **interfejs użytkownika** (np. prosty CLI lub wizualizacja drzewa stanów), ułatwiający analizę działania systemu.

Dalsze prace mogą również obejmować analizę złożoności obliczeniowej zaproponowanych algorytmów oraz ewentualne optymalizacje, w szczególności w zakresie generowania dekompozycji i eksploracji przestrzeni stanów. Interesującym kierunkiem rozszerzeń byłoby także dopuszczenie bardziej złożonych form akcji (np. hierarchicznych) lub integracja z istniejącymi systemami planowania.

Podsumowując, zaproponowany model stanowi solidną podstawę zarówno do implementacji, jak i dalszych badań nad reprezentacją wiedzy w dynamicznych systemach współbieżnych.